

Preface

About SunFounder

SunFounder is a technology company focused on Raspberry Pi and Arduino open source community development. Committed to the promotion of open source culture, we strive to bring the fun of electronics making to people all around the world and enable everyone to be a maker. Our products include learning kits, development boards, robots, sensor modules and development tools. In addition to high quality products, SunFounder also offers video tutorials to help you make your own project. If you have interest in open source or making something cool, welcome to join us!

About This Kit

The kit is suitable for the Raspberry Pi model B+, 2 model B and 3 model B.

In this book, we will show you how to build the smart car via description, illustrations of physical components, and schematic diagrams of circuits, in both hardware and software respects. You may visit our website www.sunfounder.com to download the related code and view the user manual on [LEARN -> Get Tutorials](#) and watch related videos under [VIDEO](#), or clone the code on our page of github.com at

https://github.com/sunfounder/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi

Free Support



If you have any **TECHNICAL questions**, add a topic under **FORUM** section on our website and we'll reply as soon as possible.



For **NON-TECH questions** like order and shipment issues, please **send an email to service@sunfounder.com**. You're also welcomed to share your projects on FORUM.

Contents

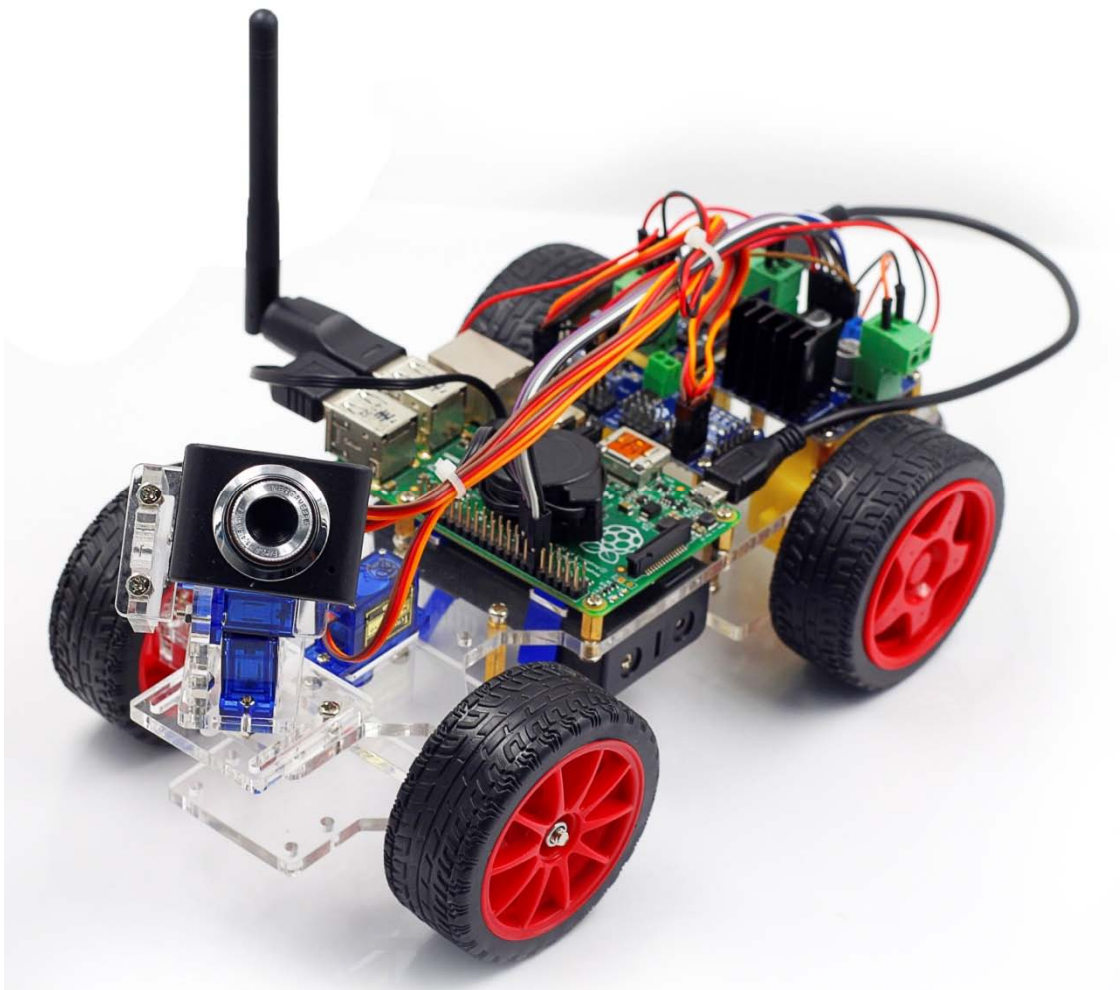
Introduction	1
Components	2
i. Acrylic Plates	2
ii. Mechanical Fasteners.....	3
iii. Drive Parts	4
iv. Electrical Components	5
v. Self-provided Parts.....	8
Assembly	9
i. Mechanical Assembly.....	9
1. Car Assembly.....	9
2. Mount Assembly	22
3. Mount + Car.....	27
ii. Electrical Module Assembly	29
iii. Circuit Connecting.....	30
Electrical Components Basics.....	40
i. Raspberry Pi.....	40
ii. Step-down DC-DC Converter Module	41
iii. Servo.....	41
iv. DC Motor Driver	42
v. USB Wi-Fi Adaptor	42
vi. Servo Controller	43
vii. 18650*2 Battery Holder.....	43
Software Related.....	44
i. Download and Install Raspbian on a TF Card.....	44
ii. Get Source Code	44
iii. Basic Software Environment.....	45
Operation on PC	45
Prepare the PC for remote	45
Operation on Raspberry Pi	46
Install python-dev, python-smbus.....	46
Setup I2C port	46
MJPEG-streamer	46
iv. Calibration.....	49

Preparation	49
Start Calibration.....	52
Motor Adjustment	52
Turning Adjustment.....	52
Mount Adjustment	53
Get on the Road!.....	54
v. Program Analysis and Explanation	58
Abstract.....	58
Introduction of Socket.....	58
1. Server	58
2. Client.....	60
Summary	62

Introduction

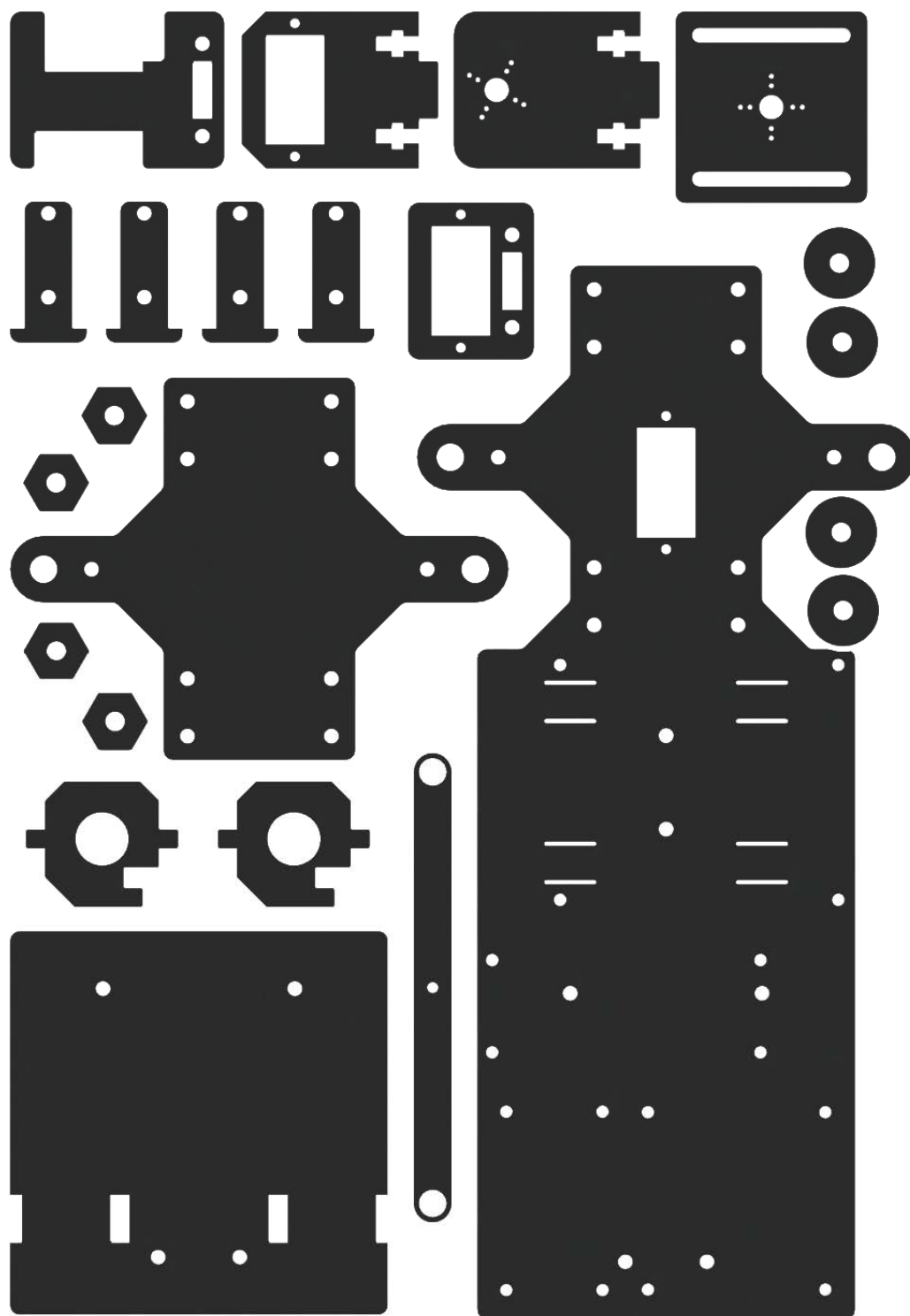
The SunFounder Smart Video Car Kit for Raspberry Pi is composed of Raspberry Pi, step-down DC-DC converter module, USB camera, DC motor driver, and PCA9685-based servo controller. From the perspective of software, the smart car is of client/server (C/S) structure. The TCP server program is run on Raspberry Pi for direct control of the car. And the video data are acquired and delivered via the open source software MJPG-streamer in a real-time manner. The TCP client program is run on PC to send the control command. Both the client and server programs are developed in Python.

The smart car is developed based on the open source hardware Raspberry Pi and integrates the knowledge of mechanics, electronics and computer, thus having profound educational significance.



Components

i. Acrylic Plates



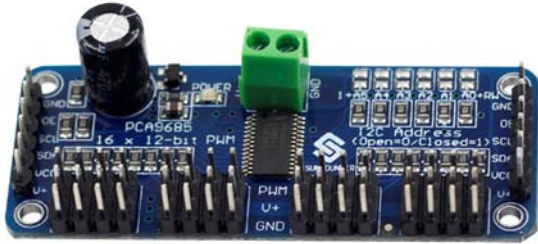
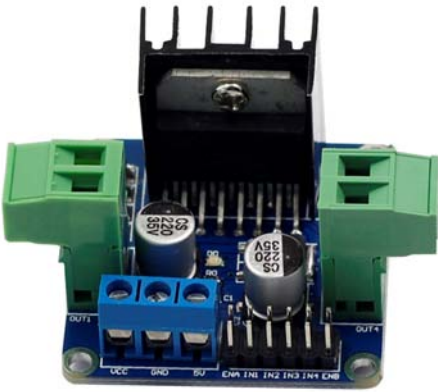
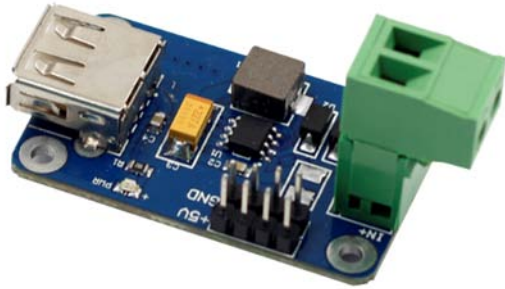
ii. Mechanical Fasteners

Parts	Name	Qty.
	M1.2*4 self-tapping screw	8
	M2*8 screw	6
	M2.5*6 screw	16
	M3*10 countersunk screw	2
	M3*8 screw	8
	M3*10 screw	6
	M3*30 screw	4
	M4*25 screw	2
	M2.5*8 copper pillar	16
	M3*24 copper pillar	8
	M2 nut	6
	M2.5 nut	16
	M3 nut	20
	M4 self-locking nut	2
	F694ZZ flange bearing	2

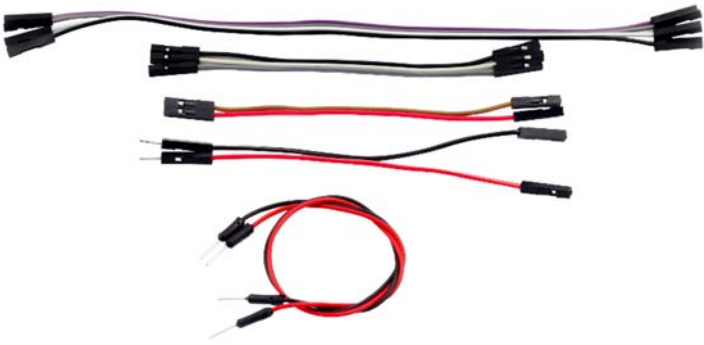
iii. Drive Parts

Parts	Name	Qty.
	Tower Pro Micro Servo SG90	3
	Gear reducer	2
	Driven wheel	2
	Active wheel	2

iv. Electrical Components

Parts	Name	Qty.
	Raspberry Pi Model B+	1
	16-Channel 12-bit PWM driver (servo controller)	1
	L298N DC motor driver	1
	Step-down DC-DC converter module	1

	USB Wi-Fi adapter	1
	USB camera	1
	18650*2 Battery holder	1
	Ribbon	1
	USB cable	1

	Cross socket wrench	1
	Cross screwdriver	1
	Heat shrink tubing	1
	20cm jumper wire (F to F)	4
	10cm jumper wire (F to F)	5
	10cm jumper wire (M to F)	2
	20cm jumper wire (M to M)	2

v. Self-provided Parts

The following parts are not included in the set.

Parts	Name	Qty. Needed
	18650 3.7V rechargeable Li-ion battery	2
	TF Card	1

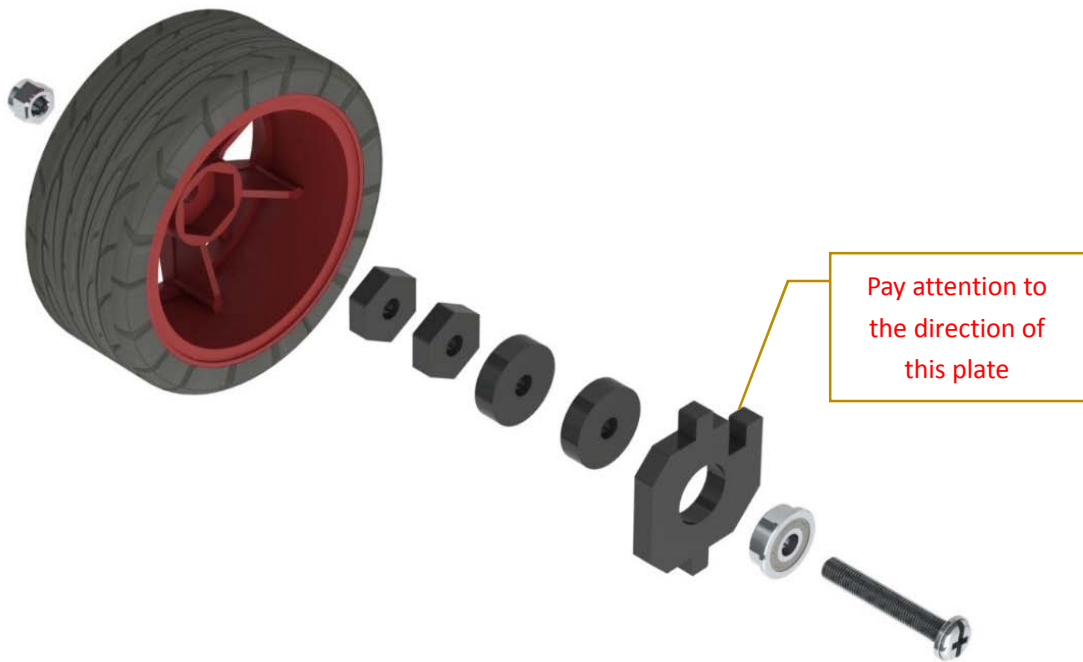
Assembly

i. Mechanical Assembly

1. Car Assembly

Front Wheels

a) Fasten the F694ZZ flange bearing, driven wheel and following acrylic plates with an M4*25 screw and an M4 self-locking nut in the way as shown below. Use the cross socket wrench to fasten the M4 self-locking nut.



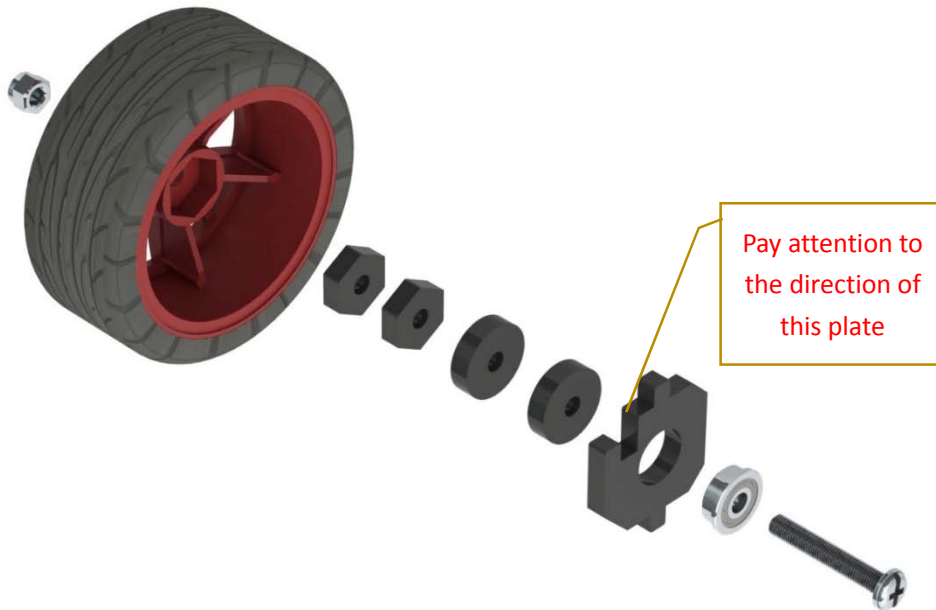
b) When completed, the assembly should look like the figure below.



c) Bear in mind that DO NOT over-tighten the nut or else the wheel cannot turn flexibly. Neither too loose, in case the gap between the parts is too large.



- a) Fasten the F694ZZ flange bearing, driven wheel and following acrylic plates with an M4*25 screw and an M4 self-locking nut in the way shown as below.



- b) When completed, the assembly should look like the figure below.



- c) Bear in mind that DO NOT over tighten the nut or else the wheel cannot turn flexibly. Neither too loose, in case the gap between the parts is too large.

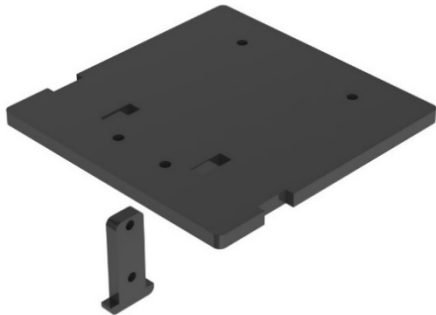


Note: The acrylic plates next to the bearing in the two wheels are of opposite faces, as shown in the following figure.

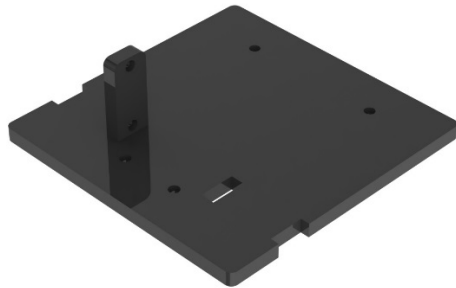


Back Half Chassis + Rear Wheels

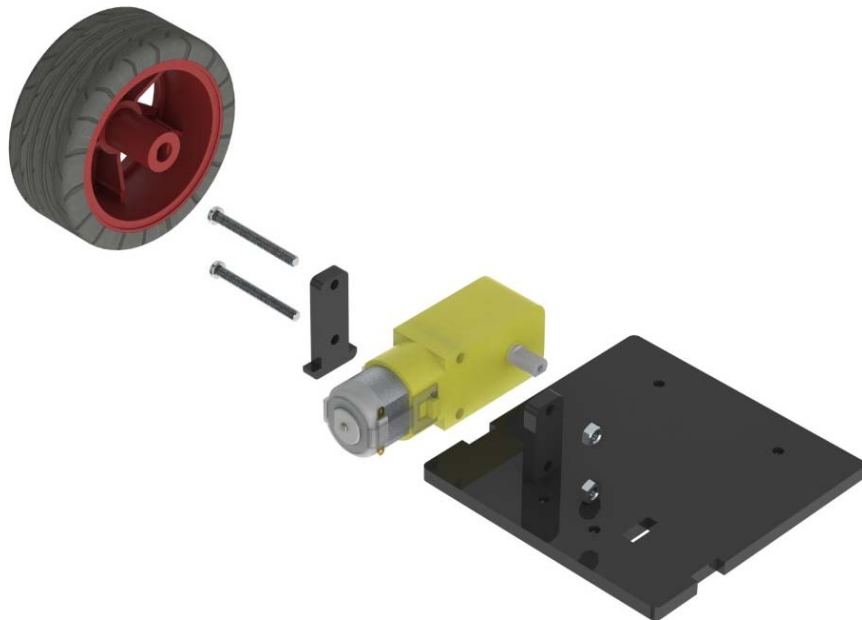
a) Assemble the following two acrylic plates



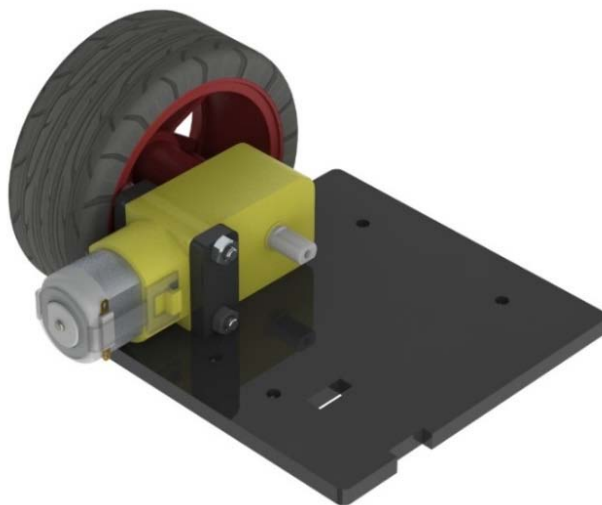
b) When completed, the assembly should look like the figure below.



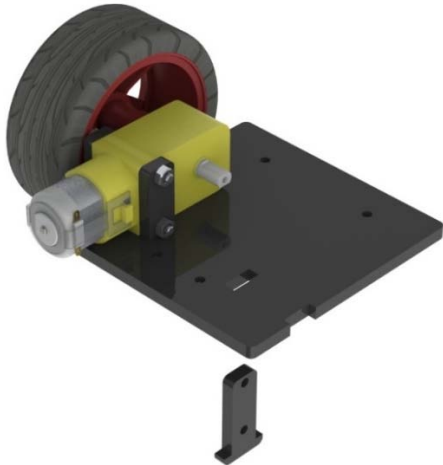
c) Assemble the gear reducer, the active wheel and following acrylic plates with two M3*30 screws and M3 nuts.



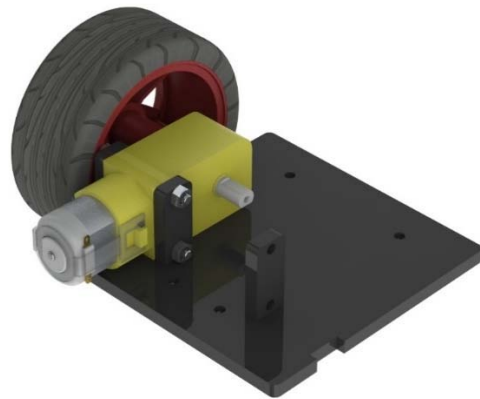
d) When completed, the assembly should look like the figure below.



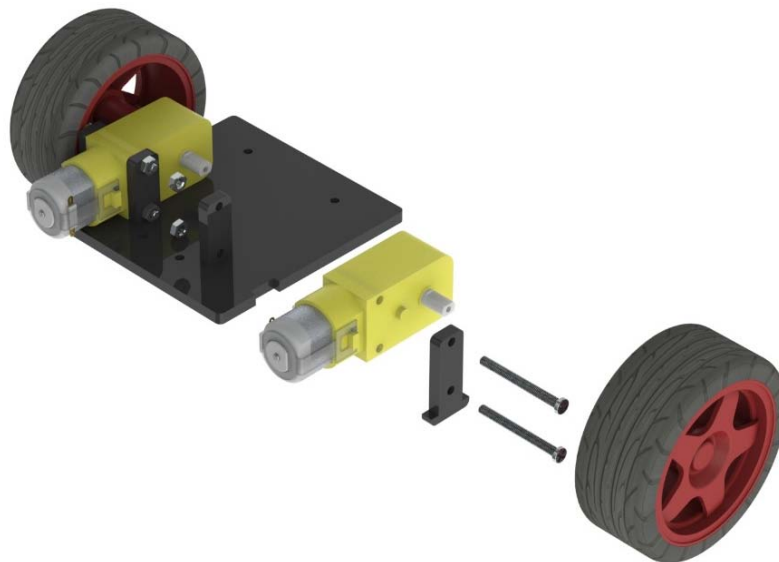
e) Assemble the following two acrylic plates



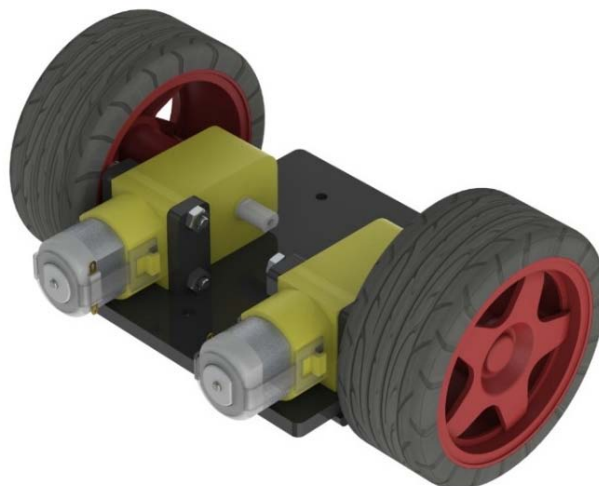
f) When completed, the assembly should look like the figure below.



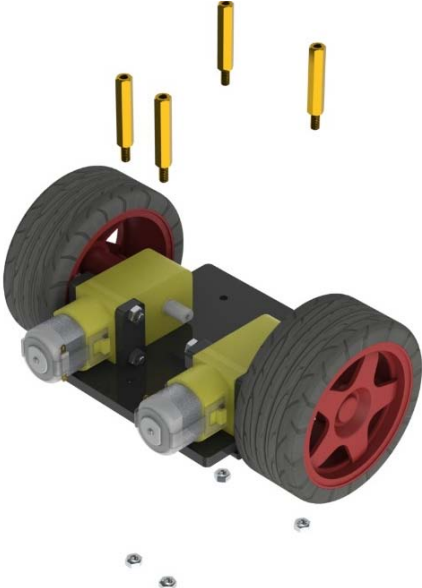
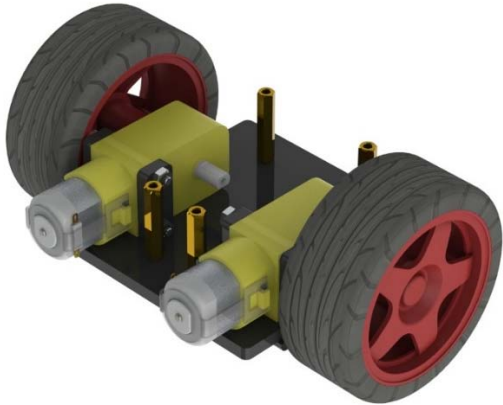
g) Assemble the gear reducer, the active wheel and the previously assembled part with two M3*30 screws and M3 nuts.



h) When completed, the assembly should look like the figure below.



Back Half Chassis + Copper Standoffs

a) Assemble 4 M3*24 copper standoffs and 4 M3 nuts into the acrylic plate part as shown below. 	b) When completed, the assembly should look like the figure below. 
---	--

Servo Adjustment

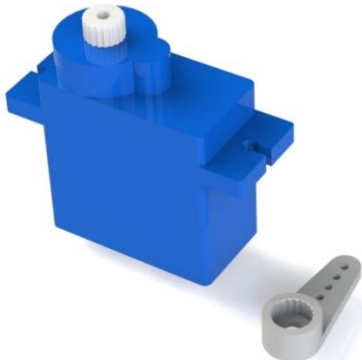
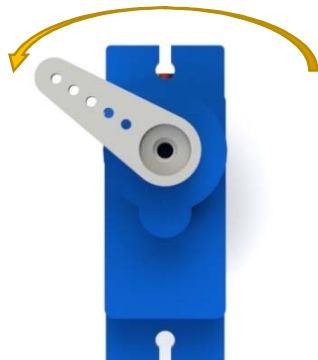
Before installing the servos, we need to adjust them to 90°. But why? The reason is that the servo we used rotates in a range of 0-180°. If the rotating in usage is beyond the allowed range, the servo may burn out. **So before assembly, we need to adjust the servo shaft to 90°**, and install the servo based on the instructions, ensuring the assembly won't go beyond limitations.

Method 1

The easiest way is to connect the servo to a Raspberry Pi and PCA9685 PWM driver board, or an Arduino board, and give the servo PWM signals so as to adjust the shaft to 90°. So it'll be done!

Method 2

There is a tricky way to do this, as the method 1 may be a little troublesome. Follow the tutorial below carefully. It may seem complex, but once you start, you will find it quite simple.

Prepare a servo and a 1-arm rocker arm. 	a) Insert the rocker arm into the servo shaft, rotate the arm counter-clockwise until it cannot move. It may be like this: 
--	--

b) It may be in the position as shown below:



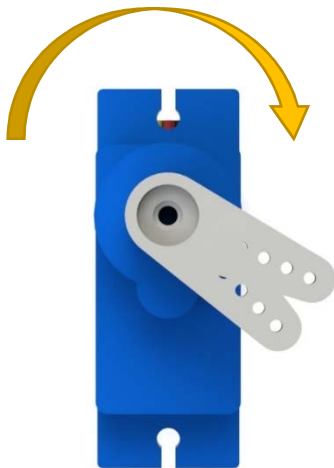
c) No matter which position the rocker arm is in, just pull it out softly, and do not rotate the shaft. Then insert it in the position as shown below, which is in the middle of the servo or a little down:



Don't worry about the precision. Fine adjustment will be made by software in calibration part. Just try to put it in an approximate position.

d) Rotate the rocker arm clockwise until it cannot move. It should be in a position close to this:

If what you get is quite different from that, you need to retake the steps above.



e) If it's almost that position, you can rotate the rocker arm counter-clockwise to the upper position.



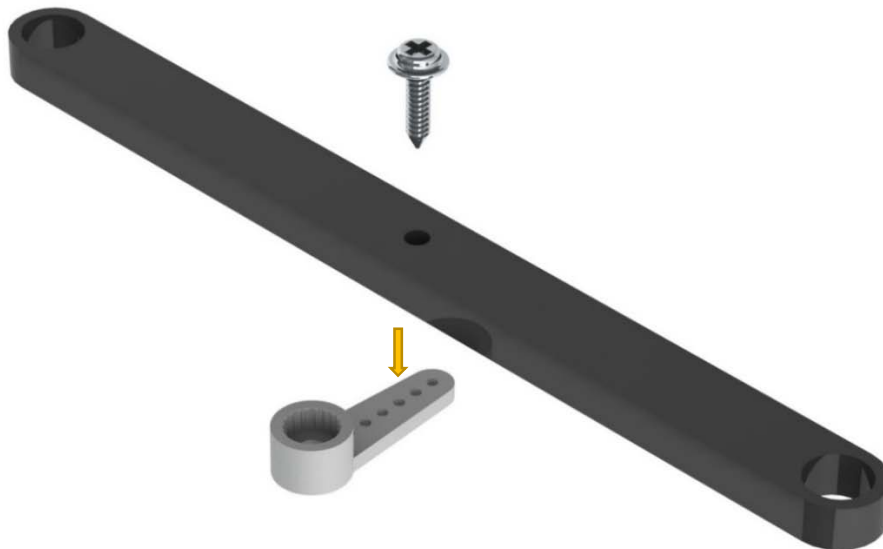
Thus the servo shaft will be in the 90° position. For fine adjustment, just leave it to related code.

At last, remove the rocker arm without moving the shaft. If the shaft moves just a little, it is acceptable. Minor mistaken movement needs no adjustments.

You're recommended to take all those three servos out from the package for adjustment prior to assembly, so the whole process will be simpler.

Steering Linkage + Servo Rocker Arm

- a) Connect the following acrylic plate to the second hole of the rocker arm (see the figure below) with an M2*8 self-tapping screw.
- ◆ The M2*8 self-tapping screw is contained in the package of the servo; it is one of the two longest screws.
 - ◆ Also the rocker arm is packaged together with the servo. *Note*: Be careful with the screw of pricking your fingers.



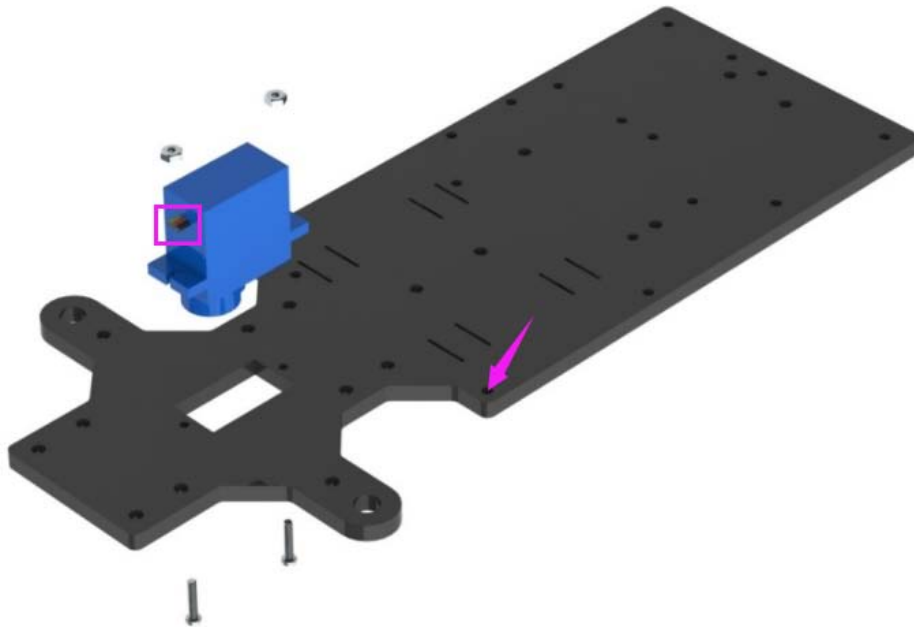
- b) Tighten the screw first, then loosen it a little to ensure the Steering Linkage's movement, the assembly should look like the figure below.



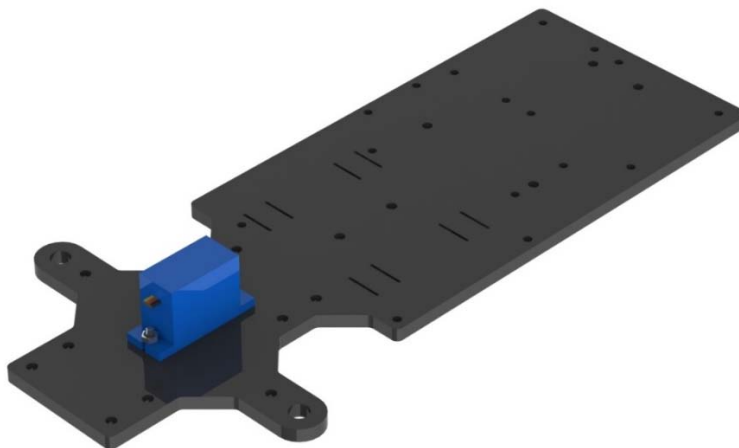
Steering Servo + Upper Plate

a) Connect the servo to the acrylic plate below with two M2*8 screws and M2 nuts.

◆ Pay attention to the face of the plate. Refer to the small hole in the plate as pointed by the arrow in the following figure.



b) When completed, the assembly should look like the figure below.



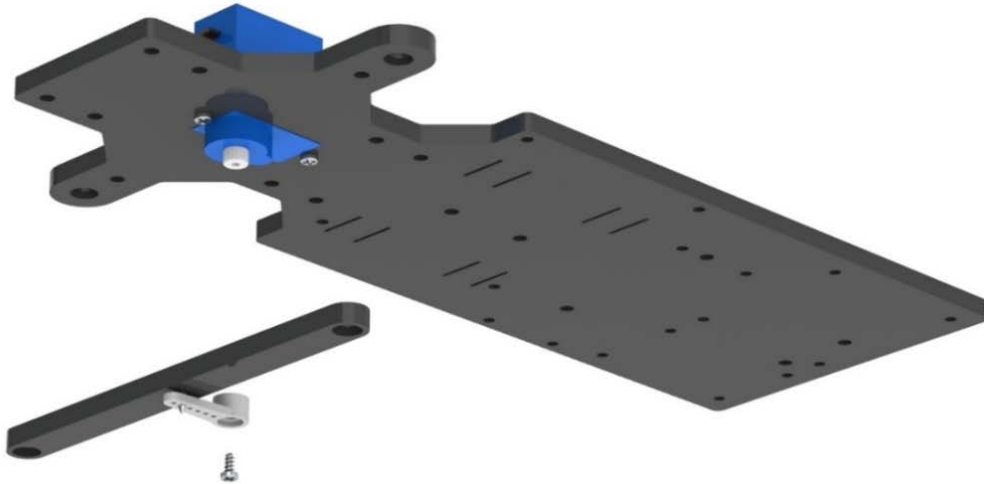
◆ **Note:** The servo should be placed with its output shaft toward the front of the plate (see the following figure).



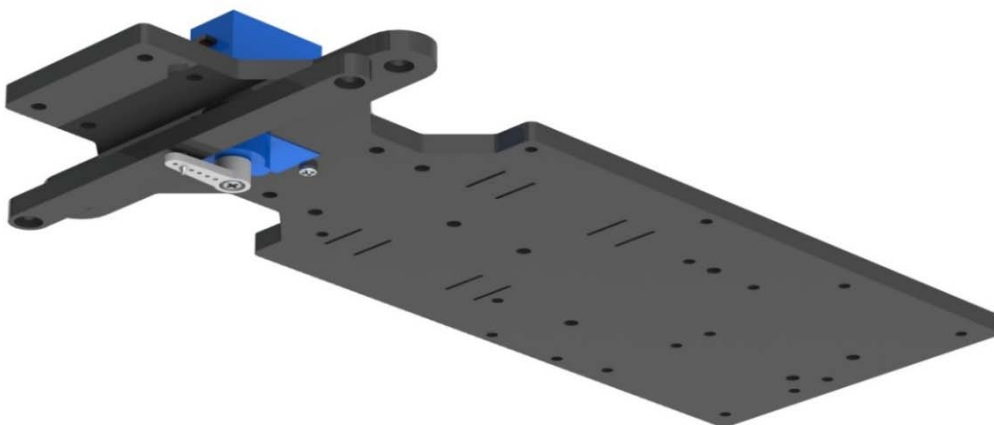
Steering Servo + Steering Linkage

a) Connect the following parts with an M2*4 screw.

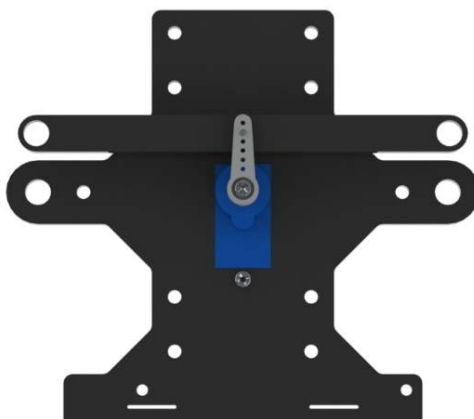
- ◆ The M2*4 screw is contained in the package of the servo; it is the shortest of the screws in the package.



b) When completed, the assembly should look like the figure below.

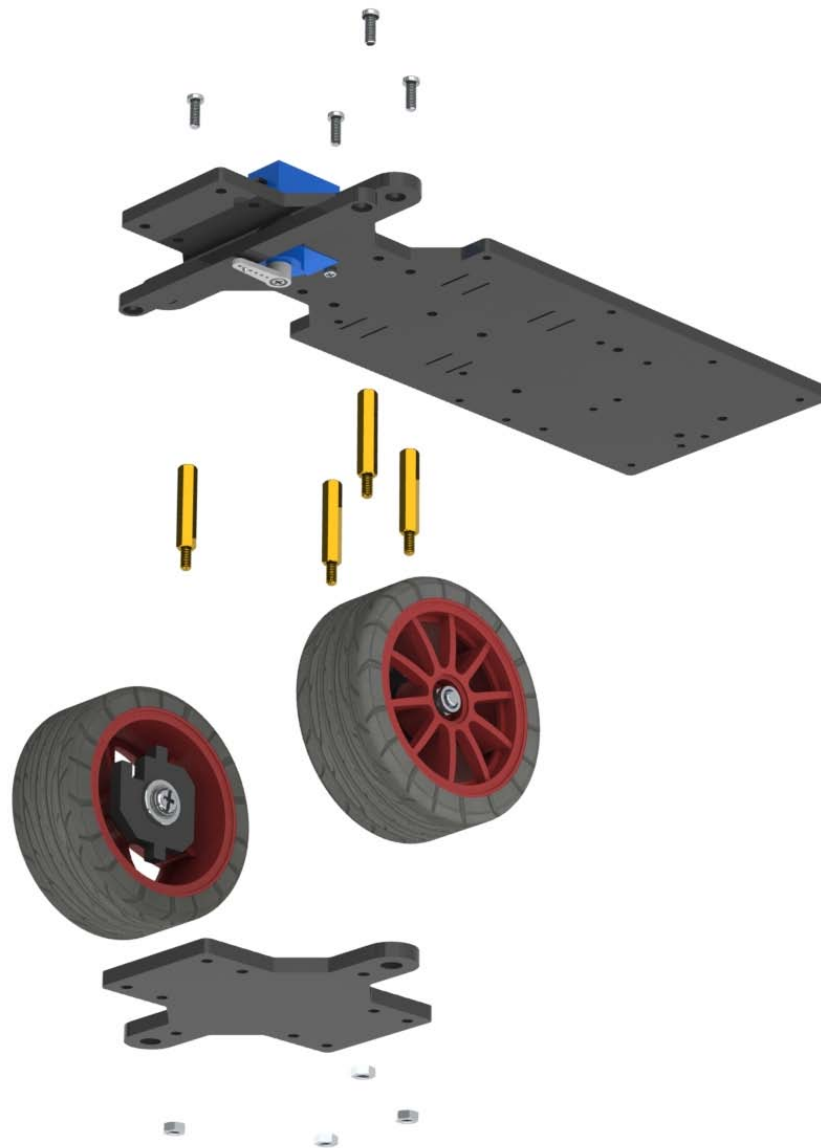


- ◆ Pay attention to the orientation during the assembly. If you find any error in the assembly, do not try to turn the axis of the servo; instead, you should disassemble it and then assemble the parts again.

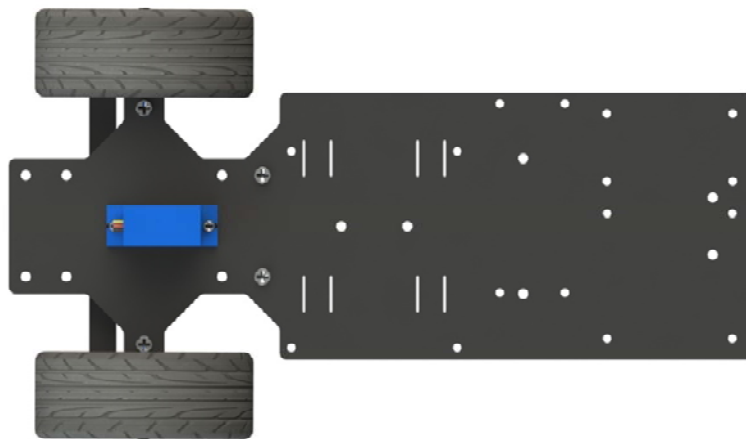


Front Chassis + Upper Plate

- a) Connect the following parts and wheels with M3*8 screws, M3*24 copper standoffs and M3 nuts, 4 for each.

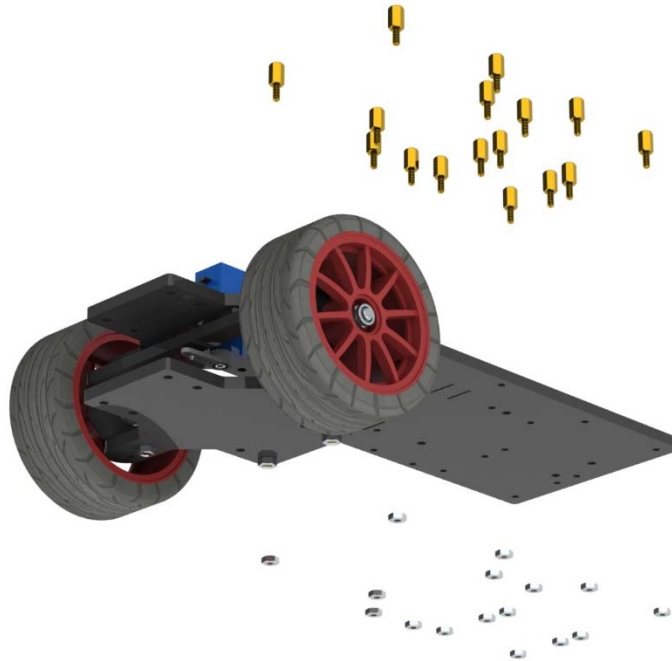


- b) When completed, the assembly should look like the figure below.

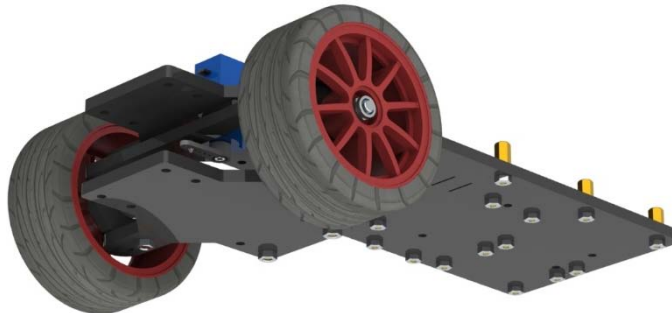


Upper Plate + Copper Standoffs

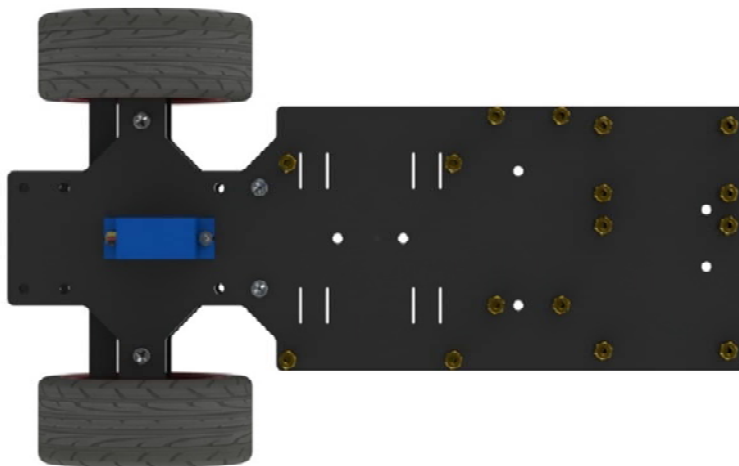
- a) Assemble 16 M2.5*8 copper standoffs and 16 M2.5 nuts into the acrylic plate as shown below.



- b) When completed, the assembly should look like the figure below.

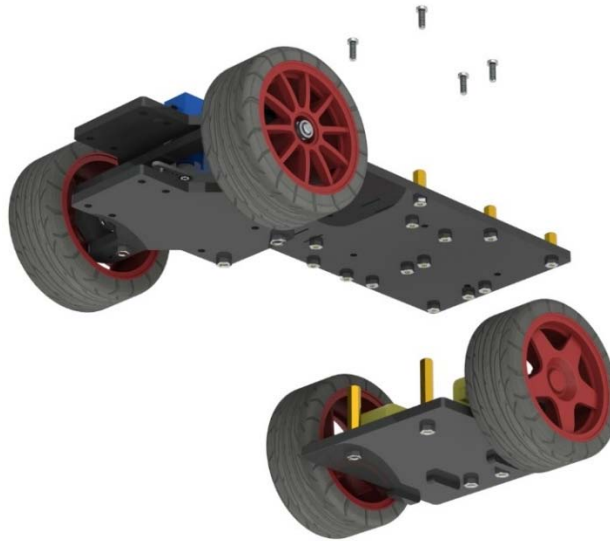


- c) The view from the back of the plate:



Back Chassis + Upper Plate

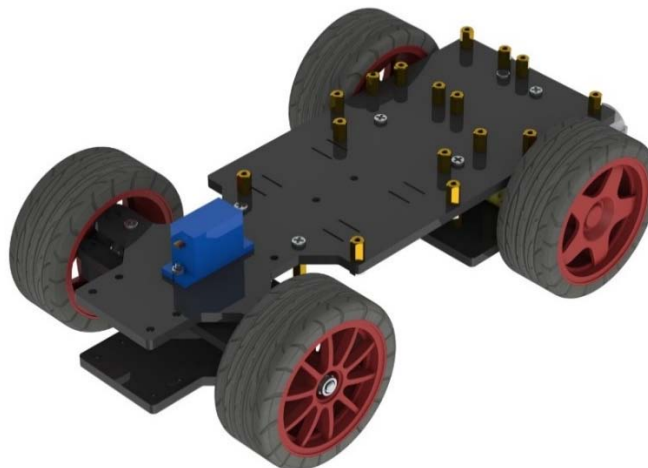
a) Connect the two assembled parts with 4 M3*8 screws.



b) When completed, the assembly should look like the figure below.



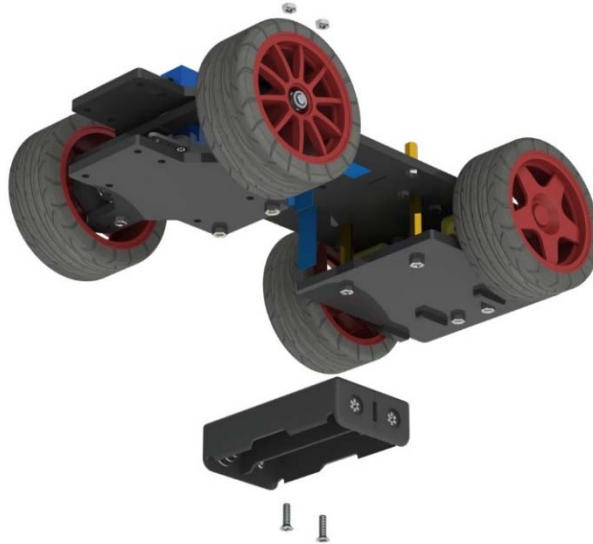
c) The view from the back of the plate:



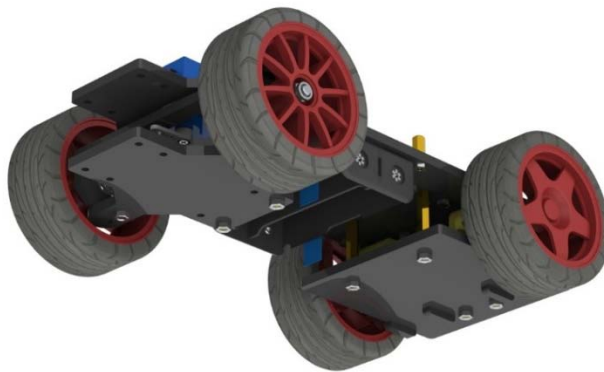
Battery Holder

a) Assemble the battery holder to the plate below with 2 M3*10 countersunk screws and 2 M3 nuts.

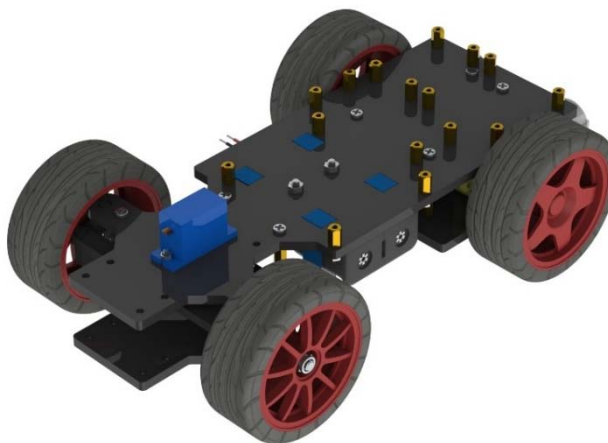
◆ You can thread a ribbon through the plate below, so it will be easy to remove the battery, which is up to you.



b) When completed, the assembly should look like the figure below.



c) The view from the top is as follows. Now the car assembly is completed.



2. Mount Assembly

Plates + Servo Rocker Arms

a) Connect the rocker arm of the servo to the acrylic plate below with 4 M1.2*4 screws.

◆ The rocker arm is packaged together with the servo.



b) When completed, the assembly should be like the figure below.

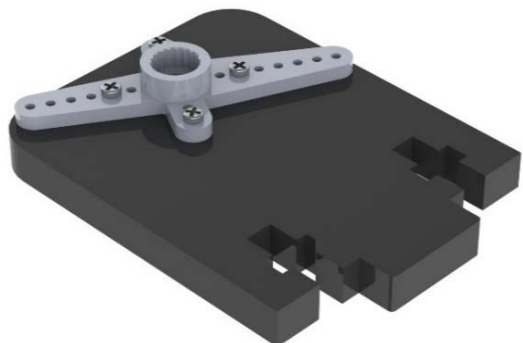


c) Connect the rocker arm of the servo to the acrylic plate below with four M1.2*4 screws.

◆ The rocker arm is packaged together with the servo.



d) When completed, the assembly should be like the figure below.

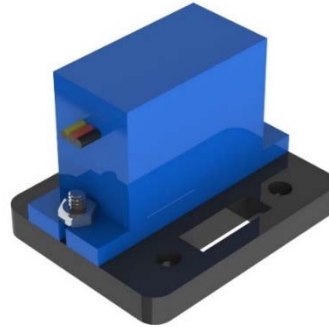


Pan/Tilt Servo + Plate

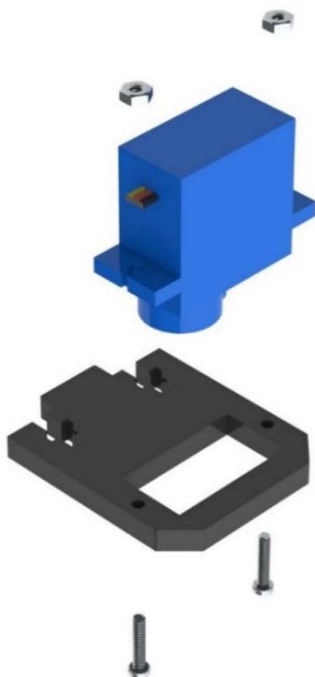
a) Connect the servo to the acrylic plate below with two M2*8 screws and M2 nuts, and we name it "**pan servo**".



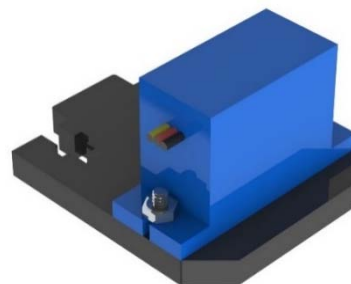
b) When completed, the assembly should look like the figure below.



c) Connect the servo to the acrylic plate below with two M2*8 screws and M2 nuts and we name it "**tilt servo**".

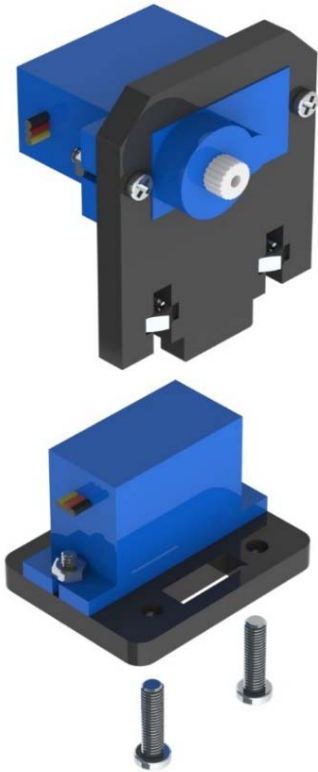


d) When completed, the assembly should look like the figure below.

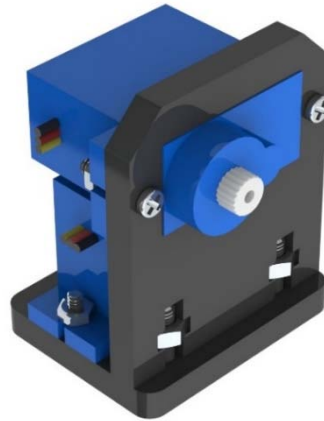


Pan Servo Plate + Tilt Servo Plate

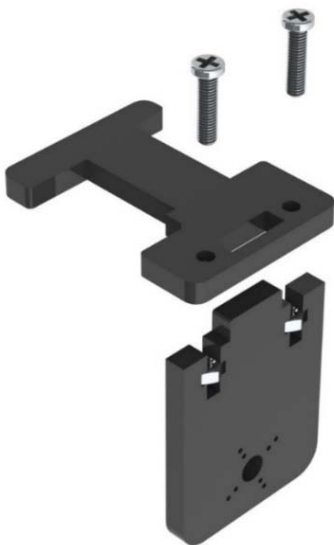
a) Connect the two plate parts together with two M3*10 screws and M3 nuts. The two plates should be perpendicular to each other.



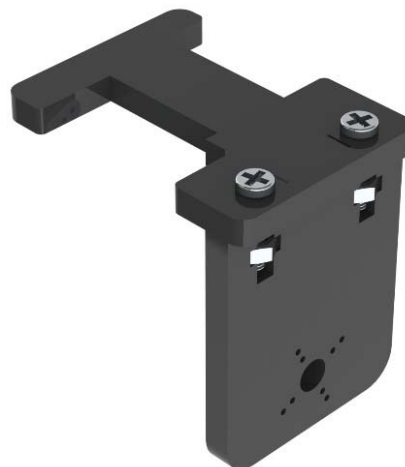
b) When completed, the assembly should look like the figure below.



c) Connect the two parts at a right angle with two M3*10 screws and M3 nuts.



d) When completed, the assembly should look like the figure below.



Servos + Rocker Arm Plates

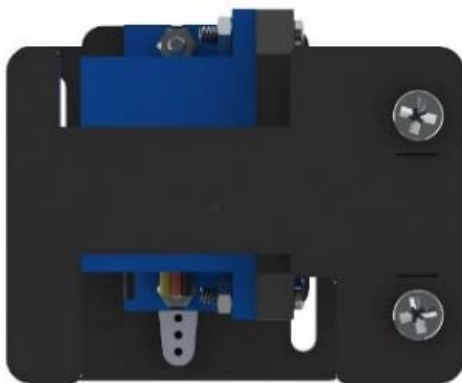
a) Connect the following parts **without any screws**.



b) When completed, the assembly should look like the figure below.



◆ Top view:

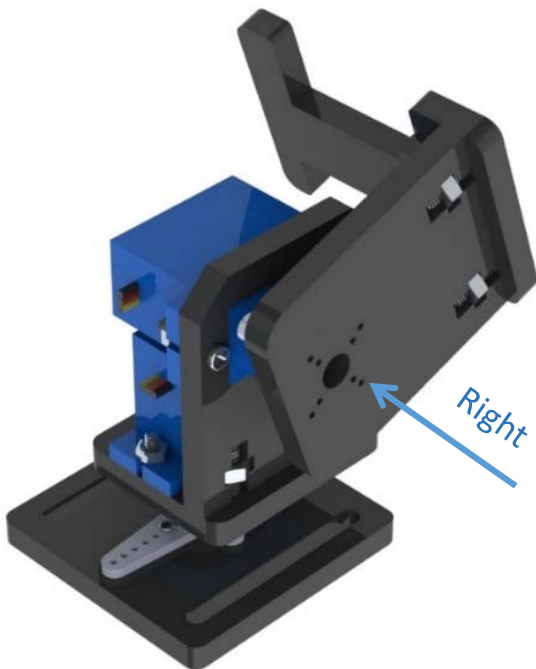


◆ Front view:



◆ Before screwing, verify the rotation angle first. Take the orientation of the tilt (top) servo shaft as the right side, as shown below. The pan (bottom) servo should be able to rotate within the range from 0 to 180 degrees, left to right; the tilt servo should rotate also from 0 to 180 degrees, front to back.

◆ Pay attention to the orientation during the assembly. If you find any error in the course, do not try to turn the axis of the servo; instead, you should disassemble it and then assemble the parts again. The calibration of the servos are the same as we've done previously in **Servo Adjustment**.



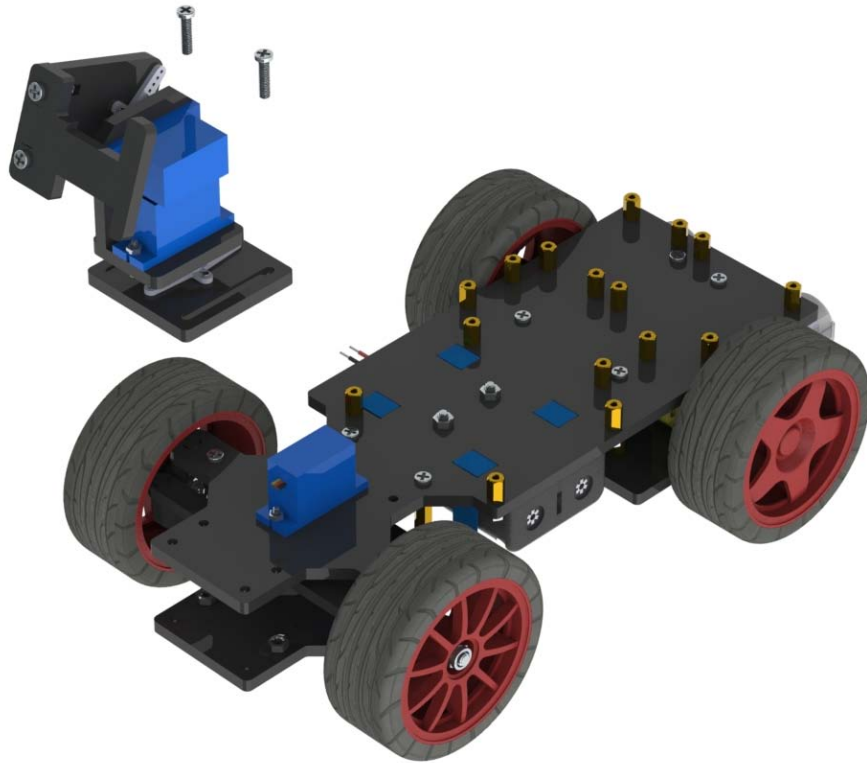
c) Then fasten them with an M2*4 screw.

The M2*4 screw is contained in the package of the servo and is the shortest of the screws.

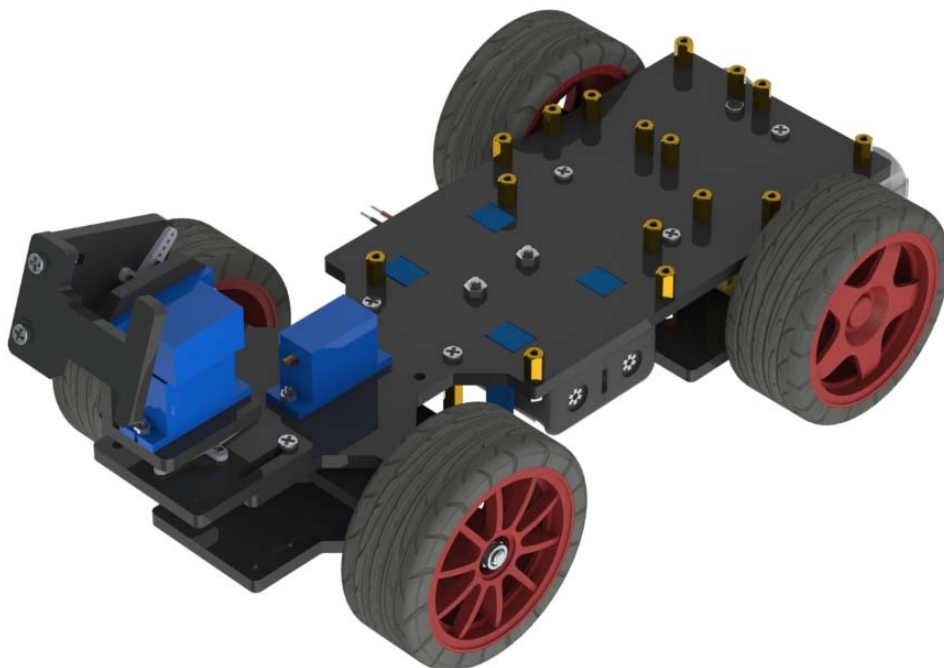


3. Mount + Car

- a) Assemble the mount part and the car with two M3*10 screws and M3 nuts.

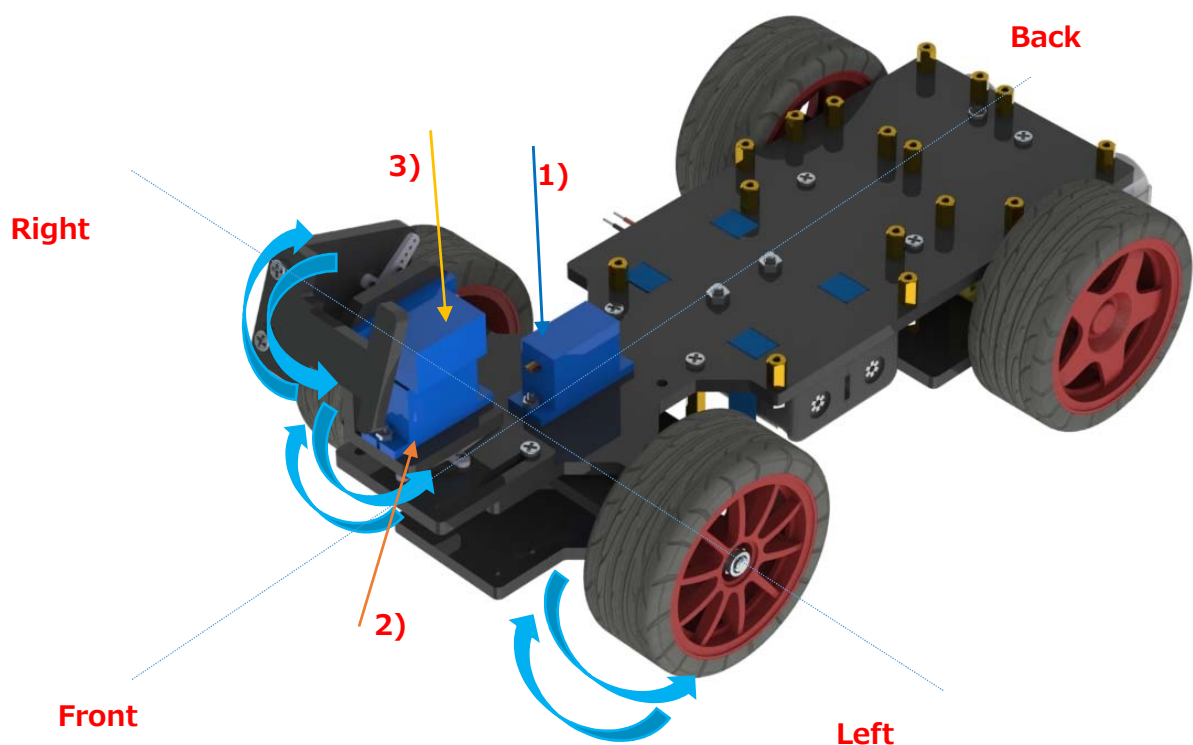


- b) When completed, the assembly should look like the figure below.



- c) Before the next assembly, you need to check the steerability of the servos.
- 1) First turn the front wheels right and left; pay attention to the utmost degree of turning: the wheels should be able to turn to a same degree towards both sides.
 - 2) Turn the pan servo of the mount part right and left to see if its turning works normally – turning precisely left to right or vice versa.
 - 3) Turn the tilt servo front to back and see if it is able to turn from precisely front to back or vice versa.

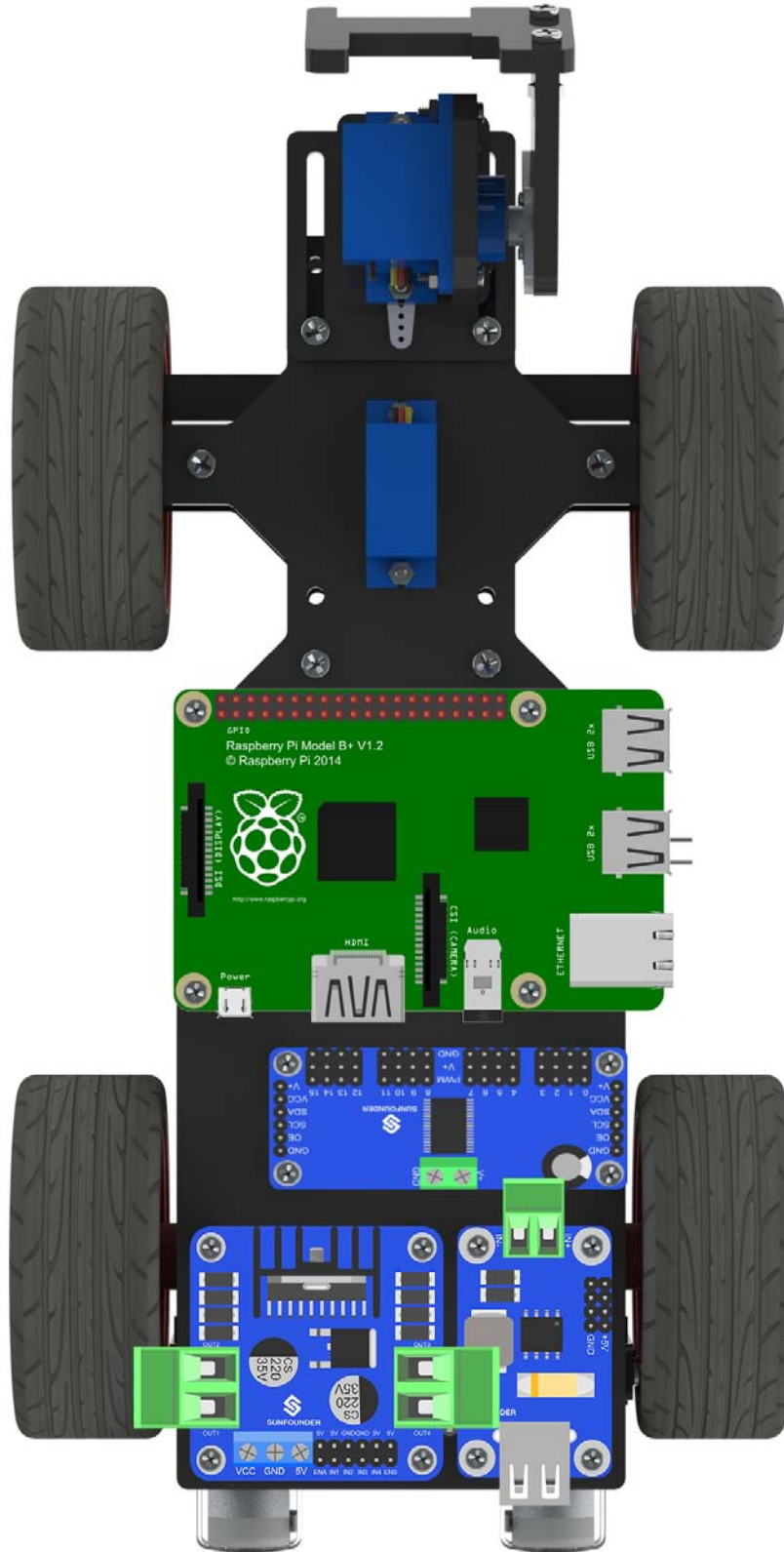
If any of the three servos fails to meet the condition, disassemble it and adjust based on **Servo Adjustment**. Please **DO** follow the instructions to adjust the servos; or else they may get burnt in subsequent operations!



ii. Electrical Module Assembly

- Assemble the electrical components to the car with M2.5*6 screws.

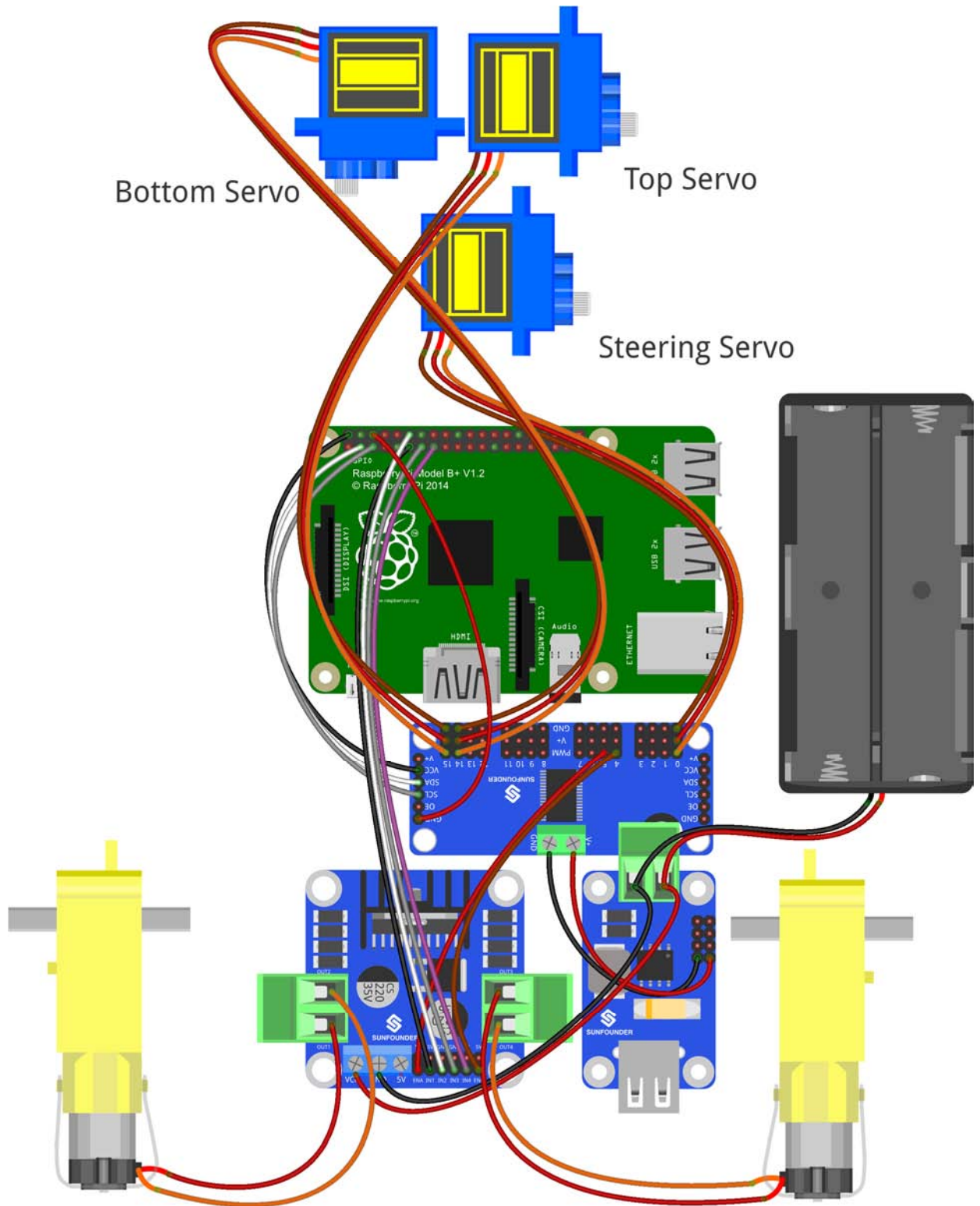
See the figure below.



iii. Circuit Connecting

Preview:

This is how it looks when all wiring is done. Looks complicated but don't worry! The detailed procedures will be given below, step by step.

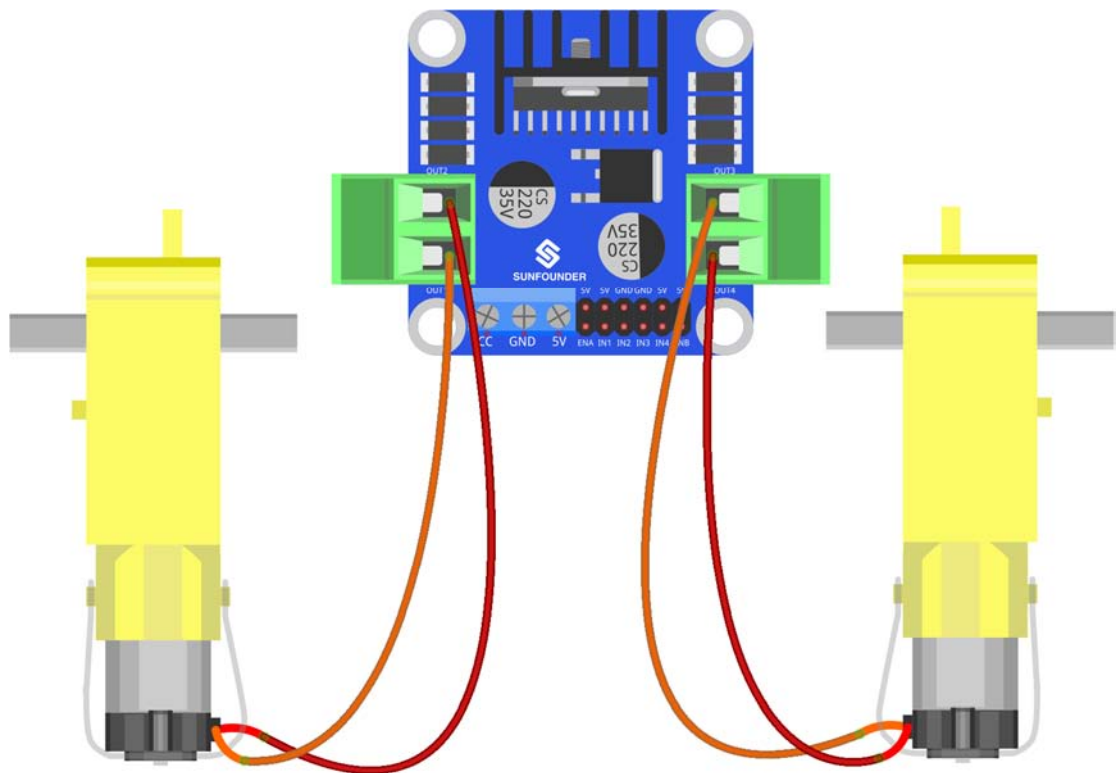


fritzing

Step 1: Connect the two DC motors with the motor driver.

You may remove the L-shaped PCB connector and plug it back after wiring.

Note: It doesn't matter how to wire the motors. After all the assembly is done, if the car moves in an opposite direction of what you control, just swap the wiring of the two motors and it will work normally.



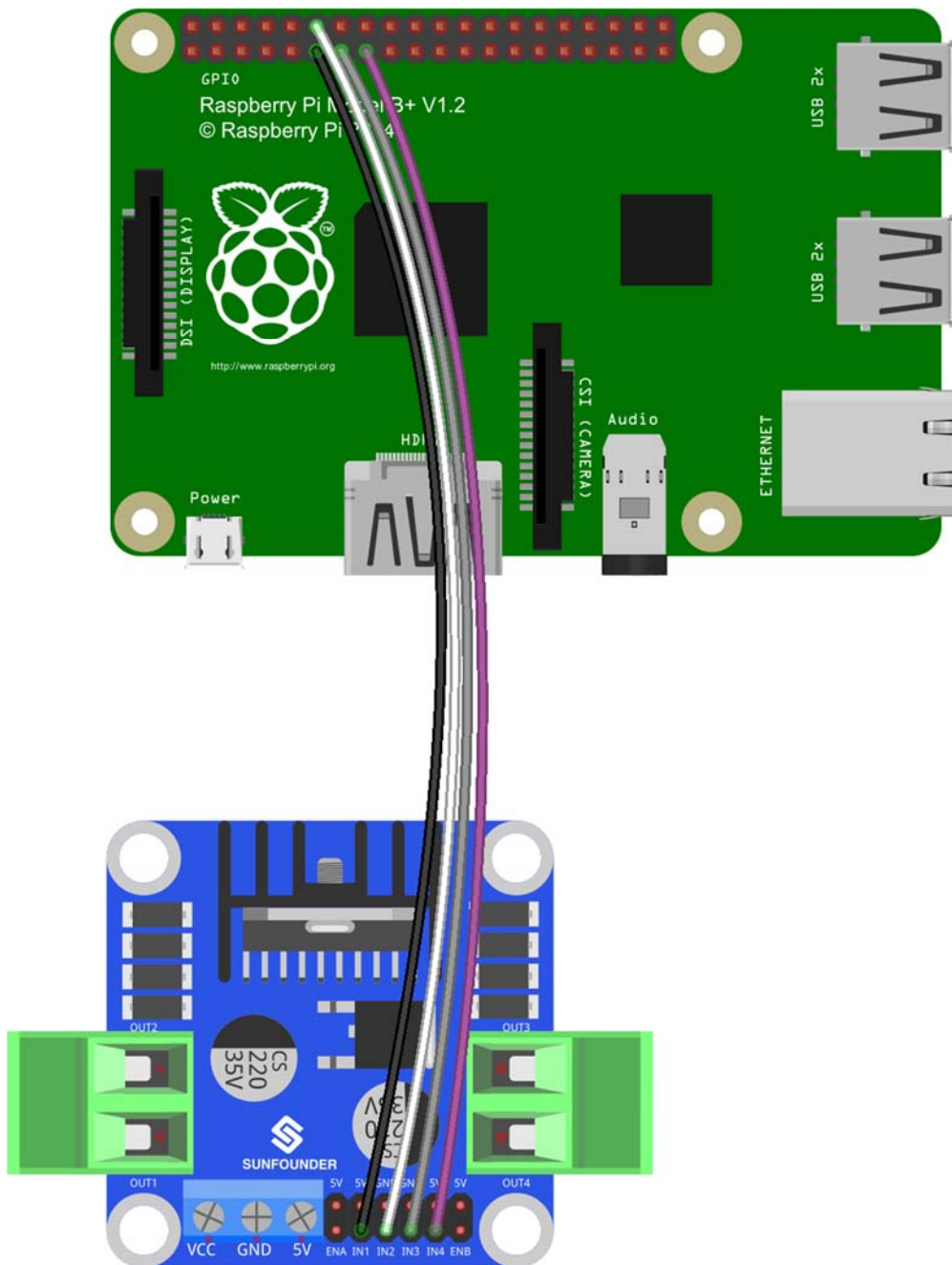
fritzing

After wiring, it should be like this:



Step 2: Connect the motor driver with the Raspberry Pi GPIO port based on the following table.

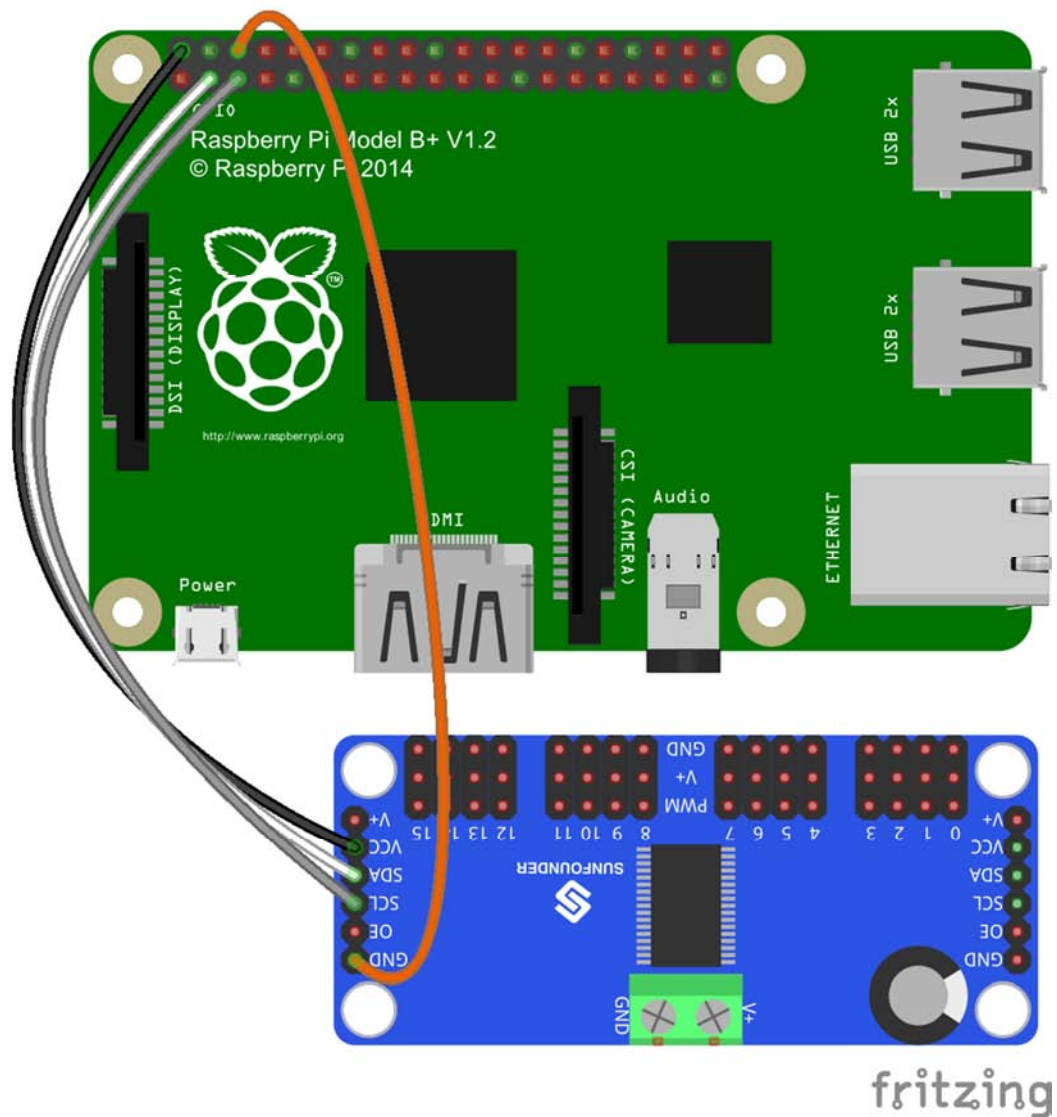
Raspberry Pi GPIO Port	DC Motor Driver
Pin11	IN1
Pin12	IN2
Pin13	IN3
Pin15	IN4



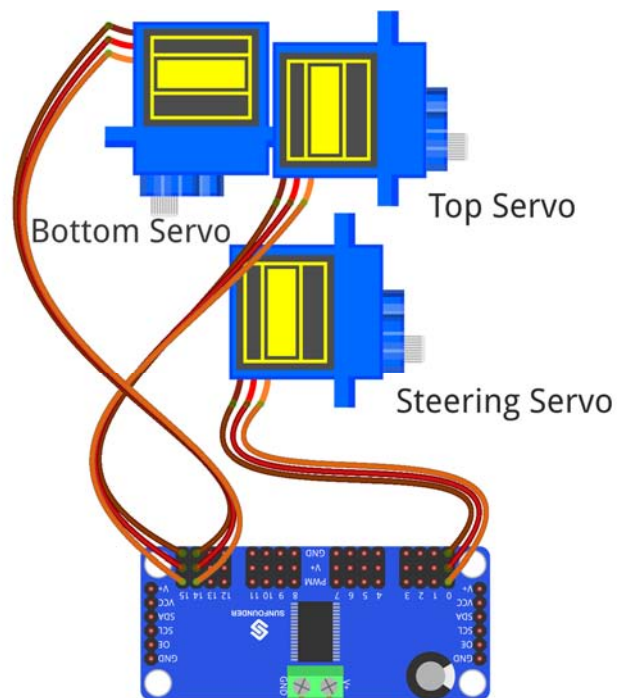
fritzing

Step 3: Connect the servo controller with the Raspberry Pi GPIO port as follows:

Raspberry Pi GPIO Port	Servo Controller
Pin 2 (5V)	VCC
Pin 3 (SDA)	SDA
Pin 5 (SCL)	SCL
GND	GND

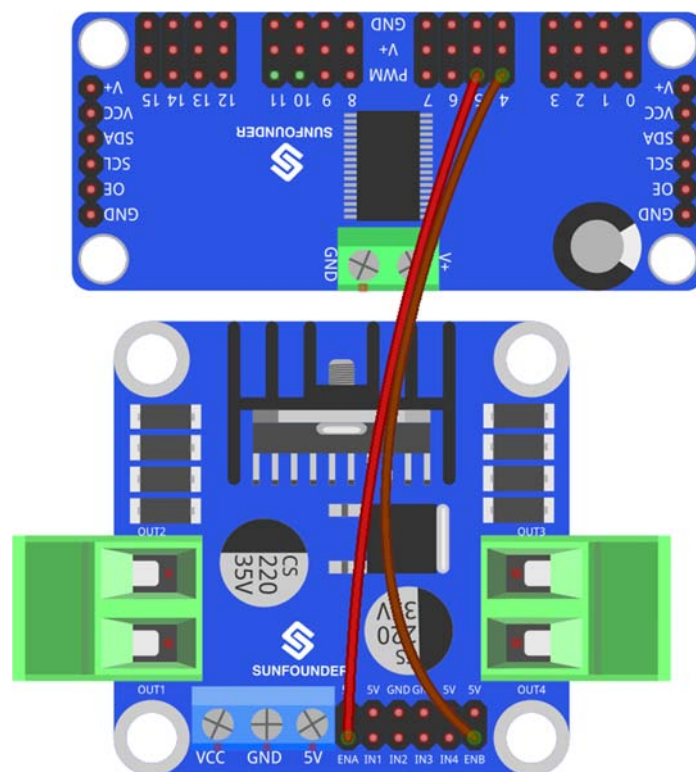


Step 4: Hook up the servo that controls the car's direction to CH0 of the servo controller, and the two servos that control the view of the camera to CH14 and CH15 respectively, as shown below:



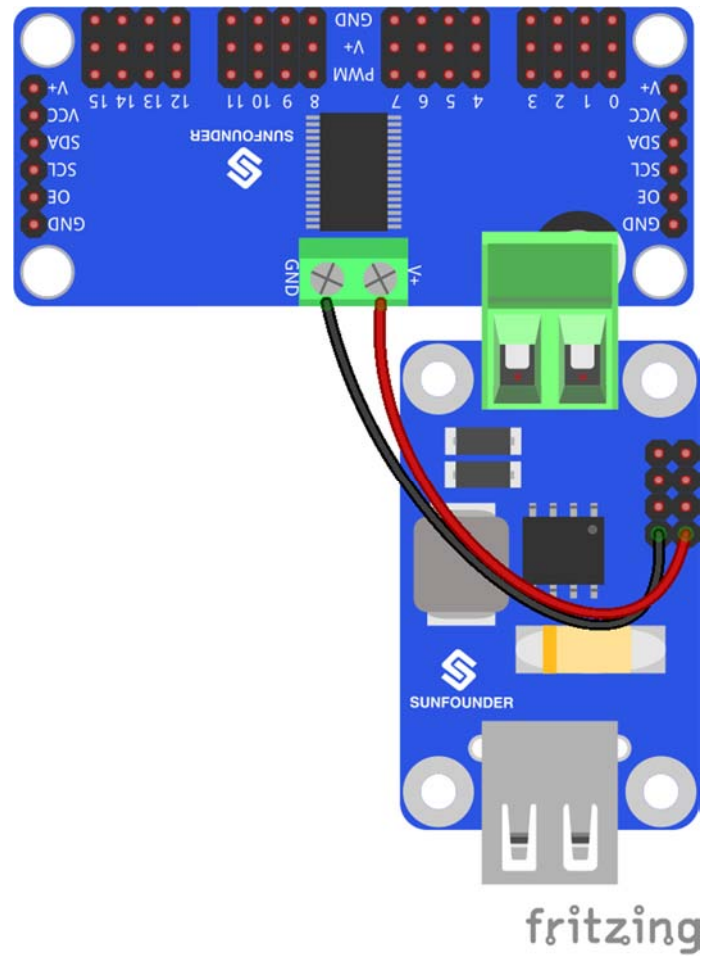
fritzing

Step 5: Connect the motor driver with the servo controller.

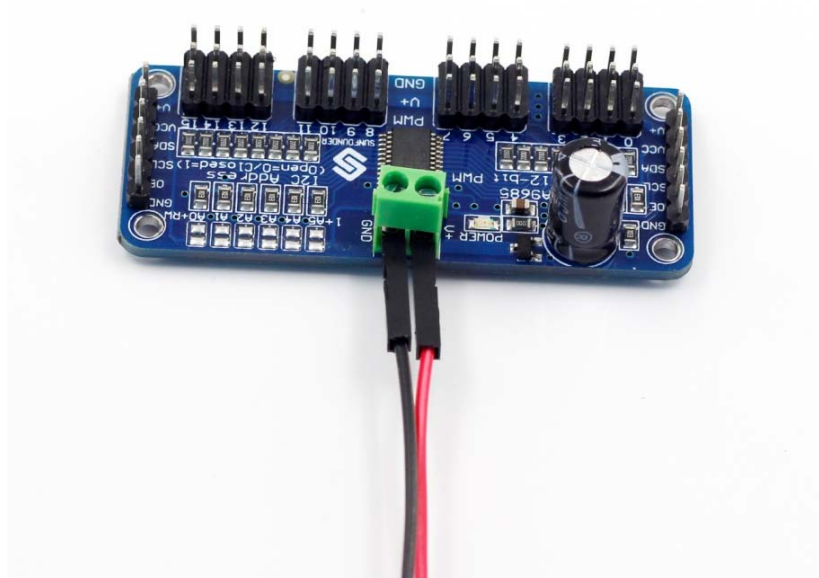


fritzing

Step 6: Connect the servo controller with the step-down DC-DC converter module. For the connector, loosen the screws, insert the wire, and then tighten the screws with a screwdriver.

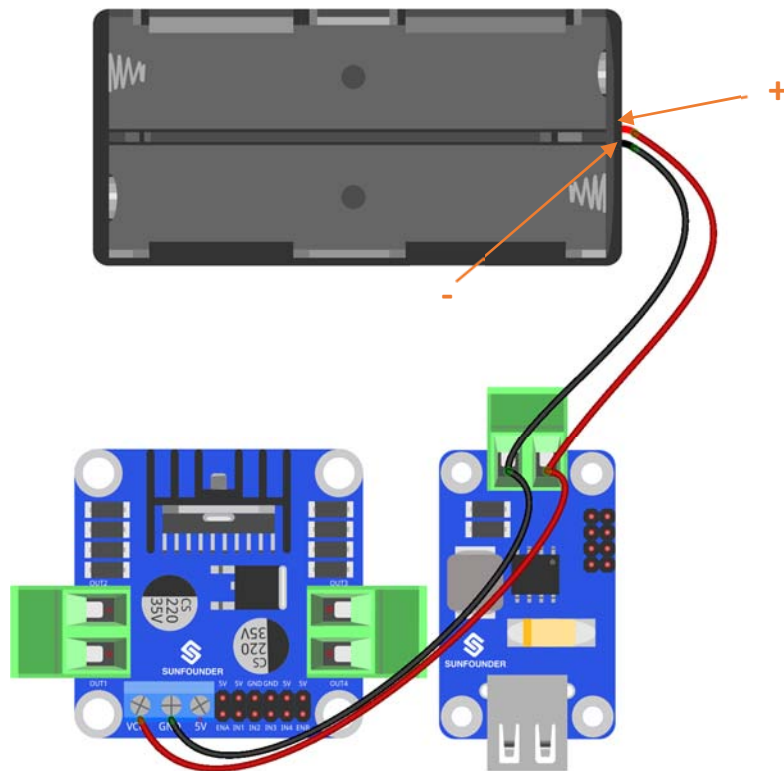


The connection should be like:



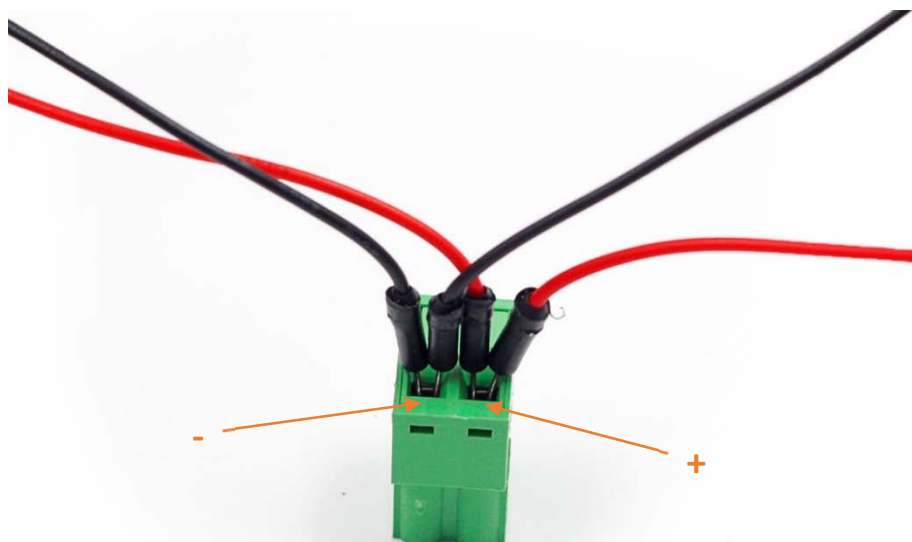
Step 7: Connect the battery holder with the step-down DC-DC converter module and the DC motor driver.

Note: Please **DO NOT** install the batteries at this step! Do it later when you've completed all the wiring. And take care that the red wire is the power and should be connected to the anode of the holder, when the black wire, i.e. the ground, to cathode.

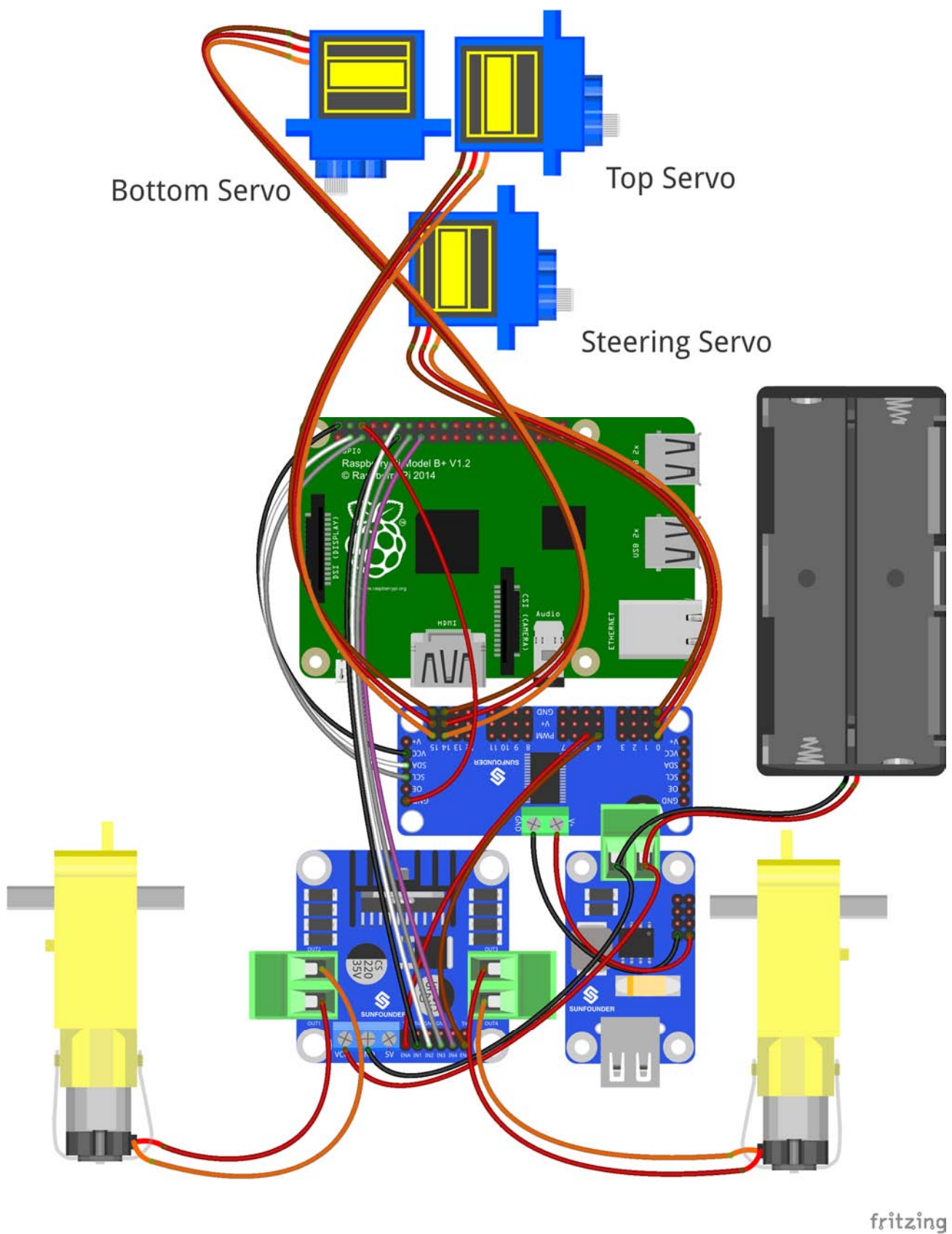


fritzing

Insert the four wires into two jacks of the connector, two in each. Pay attention: the red wires to the anode of the step-down module, when the black ones to the cathode. Also **DO NOT** plug a red wire with a black one into the same jack, or it'll cause a short circuit!



The whole picture of wiring should be like this:

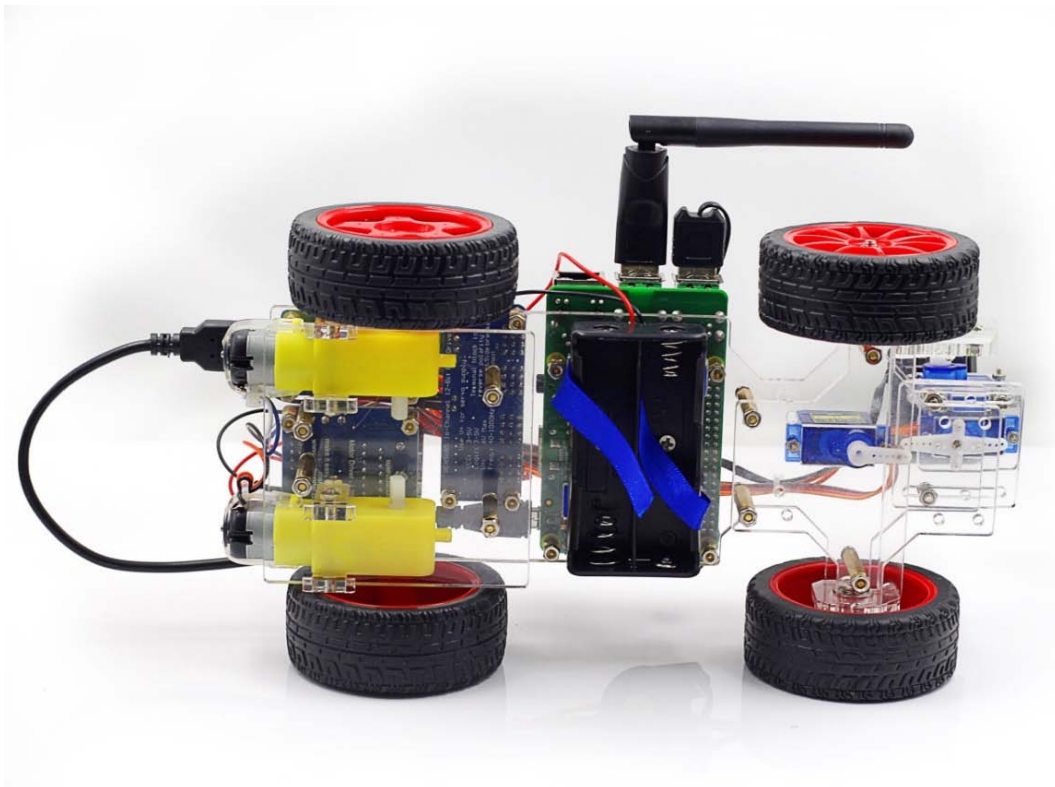
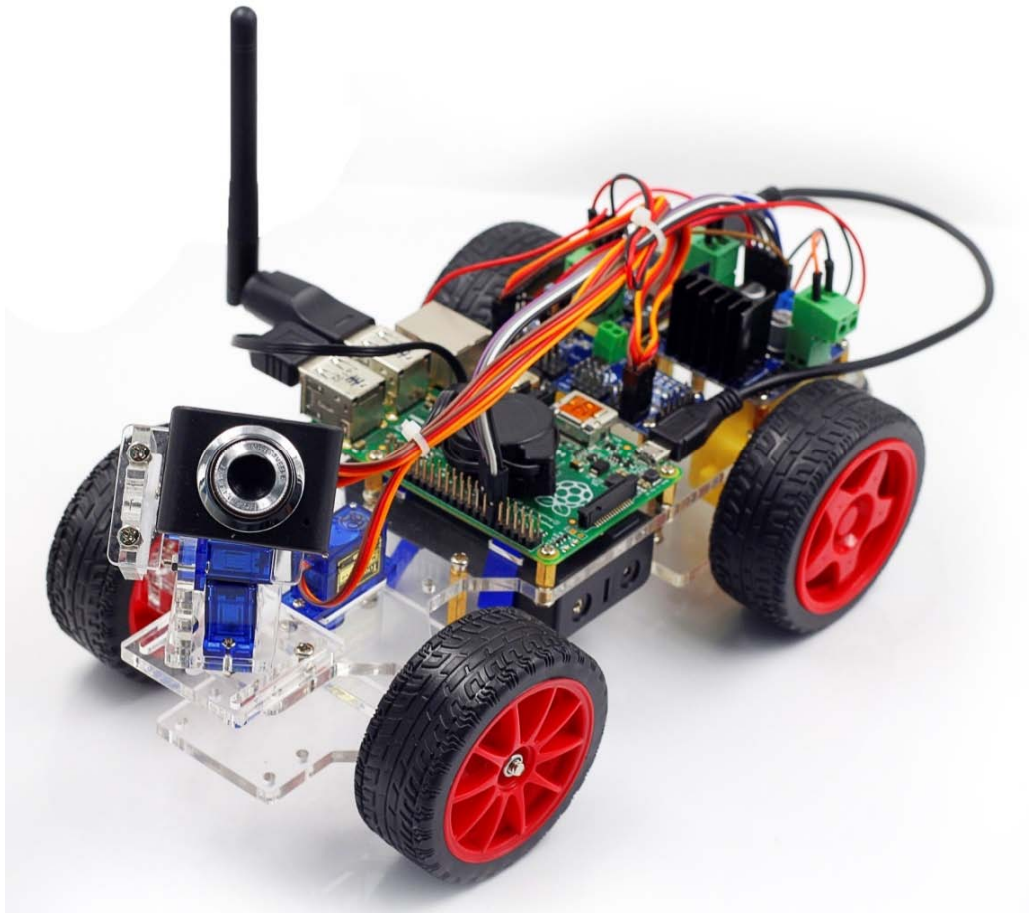


Step 8: Connect the Raspberry Pi with the step-down DC-DC converter module, the USB Wi-Fi adapter and the USB camera.



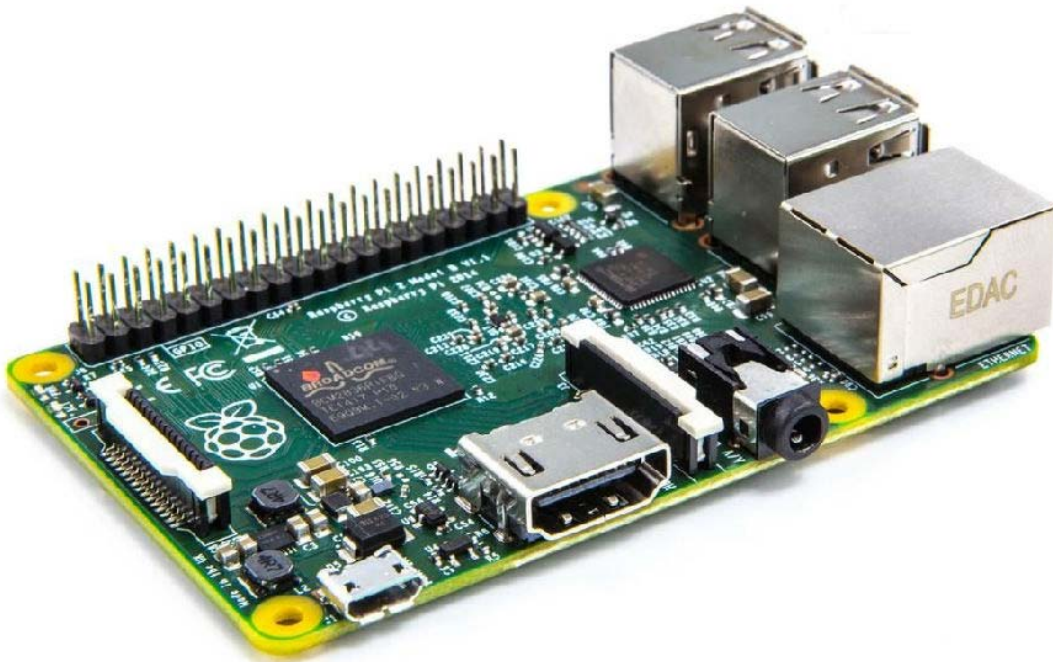
----- Now the circuit is completed. Congratulations! -----

The car should be assembled successfully as shown below:



Electrical Components Basics

i. Raspberry Pi

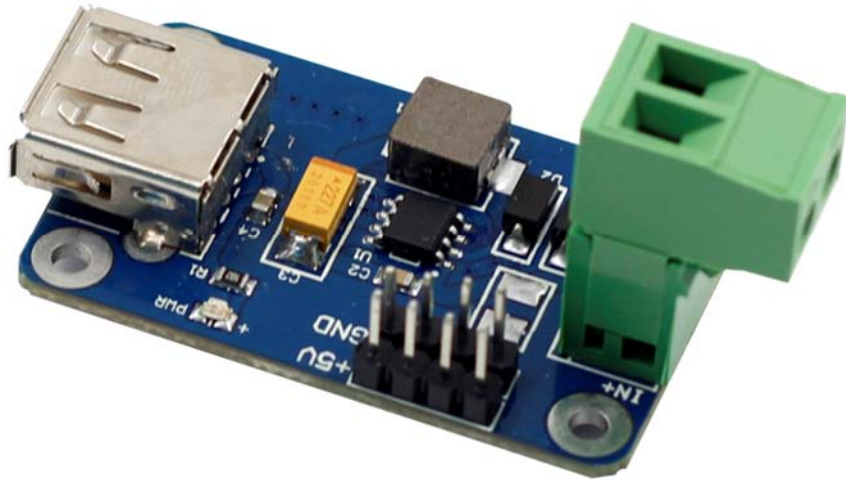


The Raspberry Pi is a low cost, credit-card sized computer that plugs into a computer monitor or TV, and uses a standard keyboard and mouse. It is a capable little device that enables people of all ages to explore computing, and to learn how to program in languages like Scratch and Python. It's capable of doing everything you'd expect a desktop computer to do, from browsing the internet and playing high-definition video, to making spreadsheets, doing word-processing, and playing games.

What's more, the Raspberry Pi has the ability to interact with the outside world, and has been used in a wide array of digital maker projects, from music machines and parent detectors to weather stations and tweeting birdhouses with infra-red cameras.

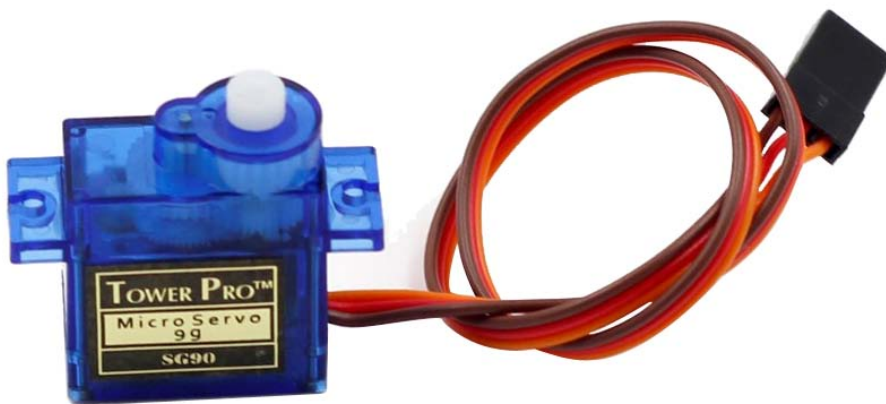
In this kit, we use Raspberry Pi as the core controller of driving equipment like DC motor and servo. With the device and a camera collaborated, video data can be acquired in a real-time manner and delivered by Wi-Fi network.

ii. Step-down DC-DC Converter Module



Built based on the chip XL1509, the module converts the battery output of 7.4V to 5V, so as to supply power to Raspberry Pi and the servo. As a DC to DC converter IC, the chip has an input voltage ranging from 4.5V to 40V and generates an output voltage of 5V with a current of as high as 2A. Please note: only when the input voltage is up to 6.5V, a 5V output can be supported.

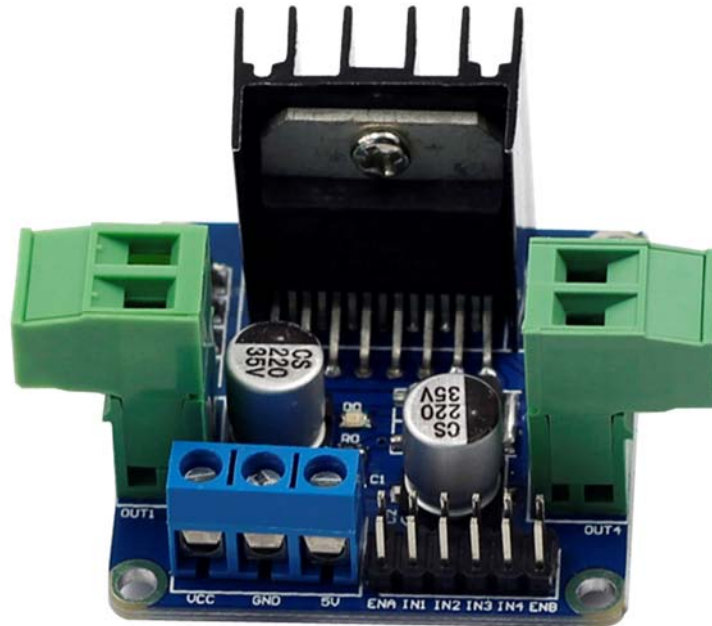
iii. Servo



In this smart car, one servo controls the direction of the car, and the other two, the movement of the camera between X axis and Y axis, thus defining the coverage of the camera. A servo is an automatic control system composed of DC motor, reduction gear set, sensor, and control circuit. It defines the rotation angle of the output shaft via delivering specific PWM signals.

Generally, a servo supports a maximum rotation angle of the shaft (like 180 degrees). It differs from a common DC motor in the rotation mode: a DC motor rotates by circle when a servo rotates in a certain degree and does not rotate in a round circle. Also, the former is used for power supply by its whole-circle rotation, while the latter is applied to controlling the rotation angle of an object (like joints of a robot).

iv. DC Motor Driver



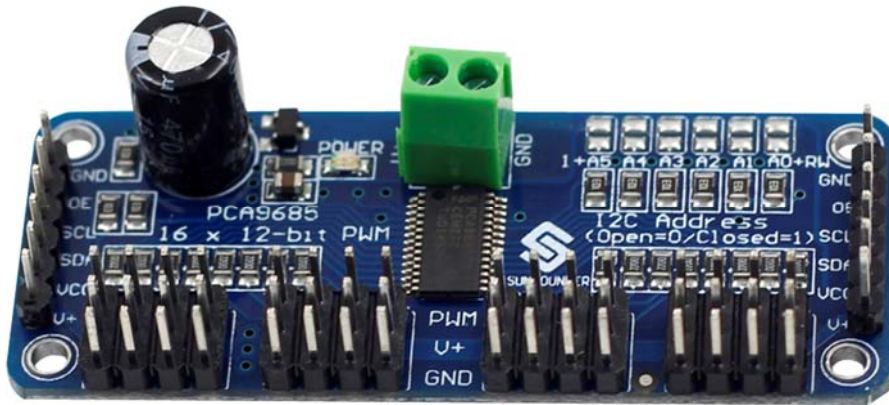
As the name suggests, the module is used to drive DC motors. The driver is built based on L298N. As a high-voltage and large-current chip for motor driving, encapsulated with 15 pins, the chip has a maximum operating voltage of 46V and an instant peak current of as high as 3A, with an operating current of 2A and rated power of 25W. Thus, it is completely capable of driving two low-power DC motors.

v. USB Wi-Fi Adaptor



The adapter helps Raspberry Pi connect to a Wi-Fi hot spot.

vi. Servo Controller



The Servo Controller is built based on PCA9685. PCA9685 is a 16-channel LED controller with I2C bus interface. The resolution ratio of each channel is 12 bits ($2^{12}=4096$ levels). The controller works in a frequency between 40Hz and 1000Hz and its duty cycle can be adjusted in a range of 0 to 100%. It provides PWM signals for the servo and controls the rotation angle of the servo. Meanwhile, the module controls the duty cycle of the square waves output from channel 14 and 15 to regulate the rotational speed of the DC motor, so as to control the speed of the car.

vii. 18650*2 Battery Holder



The battery is to use two 18650 batteries to power the modules and servos on the car.

Note: Please pay attention to the cathode (-) and anode (+) marks inside the holder (near each pole). Install the 18650 batteries accordingly: battery cathode to holder -, and battery anode to holder +.

Software Related

i. Download and Install Raspbian on a TF Card

If you've already installed the Raspbian system, please skip this step.

Search out the installation guide for Raspbian on the Raspberry Pi website at <https://www.raspberrypi.org/downloads/>. Then download the Raspbian to your TF card and install it.

After the installation, you may need some basic settings for the Raspberry Pi. Check out the guide for the setting on our website www.sunfouner.com.

ii. Get Source Code

Step 1. Download the source code directly from Github to your Raspberry Pi.

```
cd ~  
git clone https://github.com/sunfouner/Sunfouner_Smart_Video_Car_Kit_for_RaspberryPi.git
```

Step 2. Download the source code directly from Github to your Linux.

Method 1

Open a terminal in your Linux. Download **git**:

For Ubuntu/Debian:

```
sudo apt-get install git-core git
```

For Redhat/Fedora:

```
sudo yum install git-core git
```

Clone the repository from github:

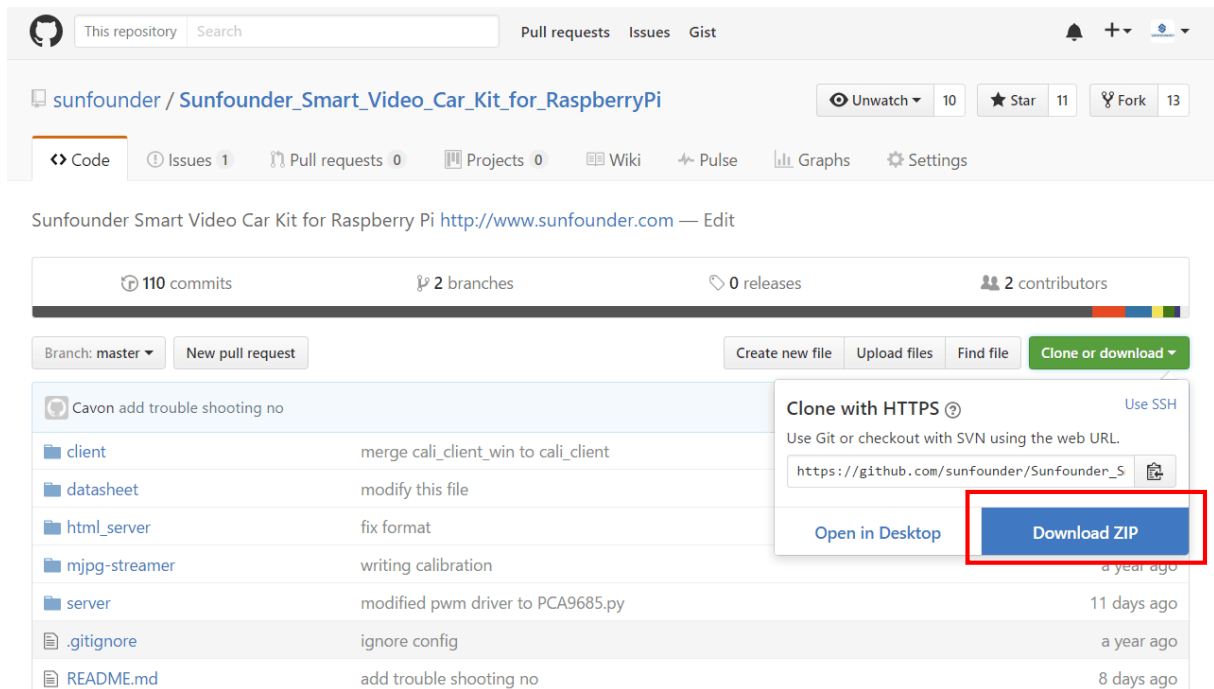
```
git clone https://github.com/sunfouner/Sunfouner_Smart_Video_Car_Kit_for_RaspberryPi.git
```

Method 2

If your Linux does not support installing **git** by **yum** or **apt-get**, download here: https://github.com/sunfouner/Sunfouner_Smart_Video_Car_Kit_for_RaspberryPi, or search for Sunfouner in Github and find the repository:

[Sunfouner_Smart_Video_Car_Kit_for_RaspberryPi](#).

Click **Download ZIP** on the sidebar of the page, as shown below.



After download,

- Go to the file, right click on it, and click **Extract here**
- Or unzip it by command:

```
unzip Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi-master.zip
```

And then, change the folder name

```
mv ./Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi-master ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi
```

iii. Basic Software Environment

Operation on PC

Prepare the PC for remote

Install `python-tk` for a remote interface

```
sudo install apt-get install python-tk
```

For Windows users:

Go to the Python website www.python.org, find the latest Python 2 and install. After installation, DO restart the computer.

Operation on Raspberry Pi

Install python-dev, python-smbus

Install `python-dev` and `python-smbus`:

```
sudo apt-get update
sudo apt-get upgrade
sudo apt-get install python-dev
sudo apt-get install python-smbus
```

Setup I2C port

Run the command to open **Raspberry Pi Software Configuration Tool (raspi-config)**

```
sudo raspi-config
```

Enable I2C:

Select **Advanced Options** => **I2C** => **<Yes>** => **<Ok>** => **<Yes>**

Select **<Finish>**. Close the window.

If a message of rebooting appears, click **<No>**. Before reboot, we still need to complete some configurations.

MJPEG-streamer

Introduction

The acquisition and transmission of video data by the SunFounder Smart Video Car is fulfilled based on MJPG-streamer.

MJPEG-streamer is a command line application that copies JPG-frame from a single input plugin to multiple output plugins. It can be used to stream JPEG files over an IP-based network from the webcam to a viewer like Firefox, Cambozola and Videolanclient or even to a Windows mobile device running the TCPMP-Player.

It was written for embedded devices with very limited resources in terms of RAM and CPU. Its origin, the "uvnc_streamer" was written, because Linux-UVC compatible cameras directly produce JPEG-data, allowing fast and performant M-JPEG streams even from an embedded device running OpenWRT. The input module "input_uvc.so" captures such JPG frames from a connected webcam.

Installation

Plug the USB camera into Raspberry Pi, and run the command `lsusb`. The GEMBIRD represents the USB camera; since it is printed on the screen, it indicates the system has recognized the camera.

```
lsusb
```

You'll see:

```
pi@raspberrypi:~ $ lsusb
Bus 001 Device 004: ID 1908:2310 GEMBIRD
Bus 001 Device 003: ID 0424:ec00 Standard Microsystems Corp. SMSC9512/9514 Fast Ethernet Adapter
```

```
Bus 001 Device 002: ID 0424:9514 Standard Microsystems Corp.  
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

Check whether the driver for the camera works normally:

```
ls /dev/vid*
```

You'll see:

```
pi@raspberrypi:~ $ ls /dev/vid*  
/dev/video0
```

If `/dev/video0` is printed, the driver is in the normal state.

Then, install the following software needed:

```
sudo apt-get install subversion  
sudo apt-get install libv4l-dev  
sudo apt-get install libjpeg8-dev  
sudo apt-get install imagemagick
```

Compile the source code of MJPG-streamer:

```
cd /home/pi/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/mjpg-streamer/mjpg-streamer  
sudo make USE_LIBV4L2=true clean all
```

Install:

```
sudo make DESTDIR=/usr install
```

Testing

Type in `sudo sh start.sh` and press Enter.

Type in the following address (replace `192.168.0.126` with your Raspberry Pi IP address) at the address bar of your browser (Firefox is recommended):

<http://192.168.0.126:8080/stream.html>

Press **Enter** and you will see the view captured by the camera displayed on the screen in a real-time manner.

MJPEG-Streamer Demo Pages

a ressource friendly streaming
application

Home

Static

Stream

Java

Javascript

VideoLAN

Control

Version info:

v0.1 (Okt 22, 2007)

Display the stream

Hints

This example shows a stream. It works with a few browsers like Firefox for example. To see a simple example click [here](#). You may have to reload this page by pressing F5 one or more times.

Source snippet

```

```



© The **MJPEG-streamer** team | Design by **Andreas Viklund**

iv. Calibration

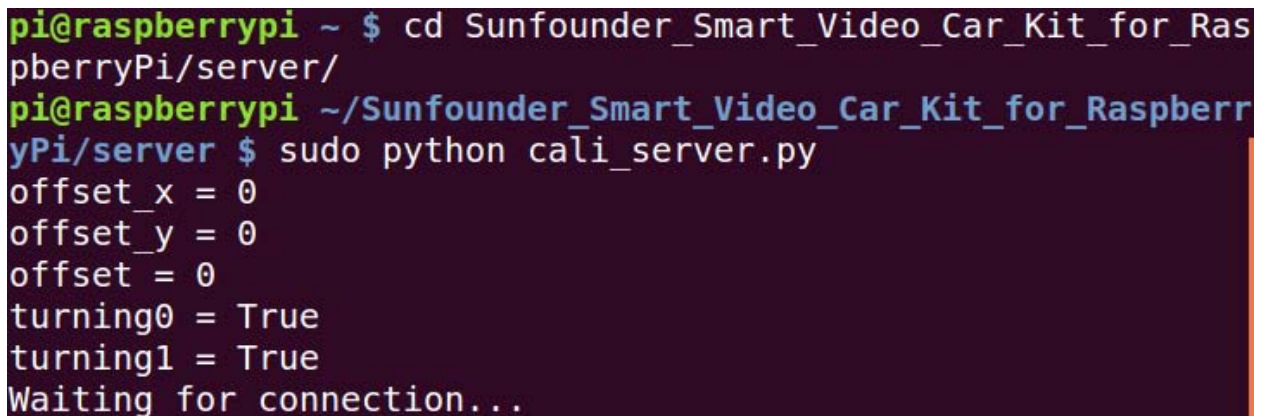
Preparation

Run the Server

Run the calibration server on the Raspberry Pi, and wait for client to connect.

Make sure that the circuit is connected properly. Power the smart car, open a terminal in Linux. Connect with your Raspberry Pi via the ssh. Go to the directory `Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/server`, run the server `cali_server.py`: if the servo gets stuck with an abnormal sound, unplug the wires at once, and redo servo adjustment for calibration in case of further damages.

```
cd ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/server
sudo python cali_server.py
```



```
pi@raspberrypi ~ $ cd Sunfounder_Smart_Video_Car_Kit_for_Ras
pberrypi/server/
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_Raspberr
yPi/server $ sudo python cali_server.py
offset_x = 0
offset_y = 0
offset = 0
turning0 = True
turning1 = True
Waiting for connection...
```

Then the contents of `config` are printed and the last line: **Waiting for connection...**

At this time, the car might move a bit (this is the original position set)

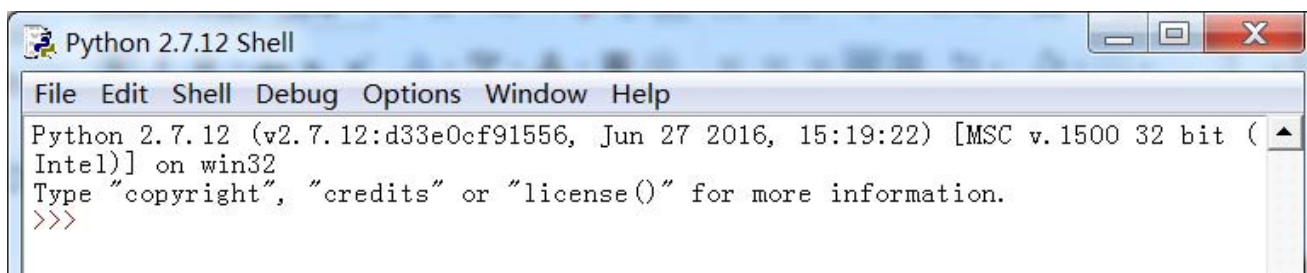
Run the Client

Run the calibration client on PC to connect the server on Raspberry Pi.

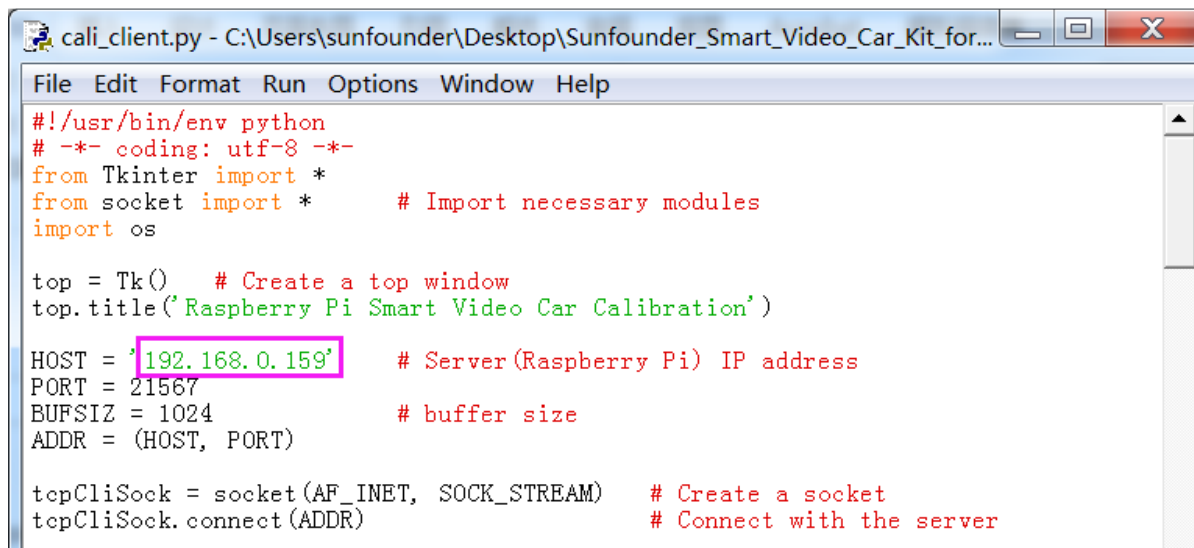
[For Windows users:](#)

Unzip the code package you downloaded.

Click the **Start** button on your computer, and type in python in the search bar, and you can find the IDLE (Python GUI). Click it and then a window will pop up.



Click **File** -> **Open** -> **Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi-master** -> **client** -> **cali_client.py** to open this file, modify the value of **HOST** for the IP address of the Raspberry Pi.



```
cali_client.py - C:\Users\sunfounder\Desktop\Sunfounder_Smart_Video_Car_Kit_for...
File Edit Format Run Options Window Help
#!/usr/bin/env python
# -*- coding: utf-8 -*-
from Tkinter import *
from socket import *      # Import necessary modules
import os

top = Tk()      # Create a top window
top.title('Raspberry Pi Smart Video Car Calibration')

HOST = '192.168.0.159'  # Server (Raspberry Pi) IP address
PORT = 21567
BUFSIZ = 1024      # buffer size
ADDR = (HOST, PORT)

topCliSock = socket(AF_INET, SOCK_STREAM)  # Create a socket
topCliSock.connect(ADDR)                  # Connect with the server
```

After modification, save the file and click the **Run** menu, and select **Run Module**.

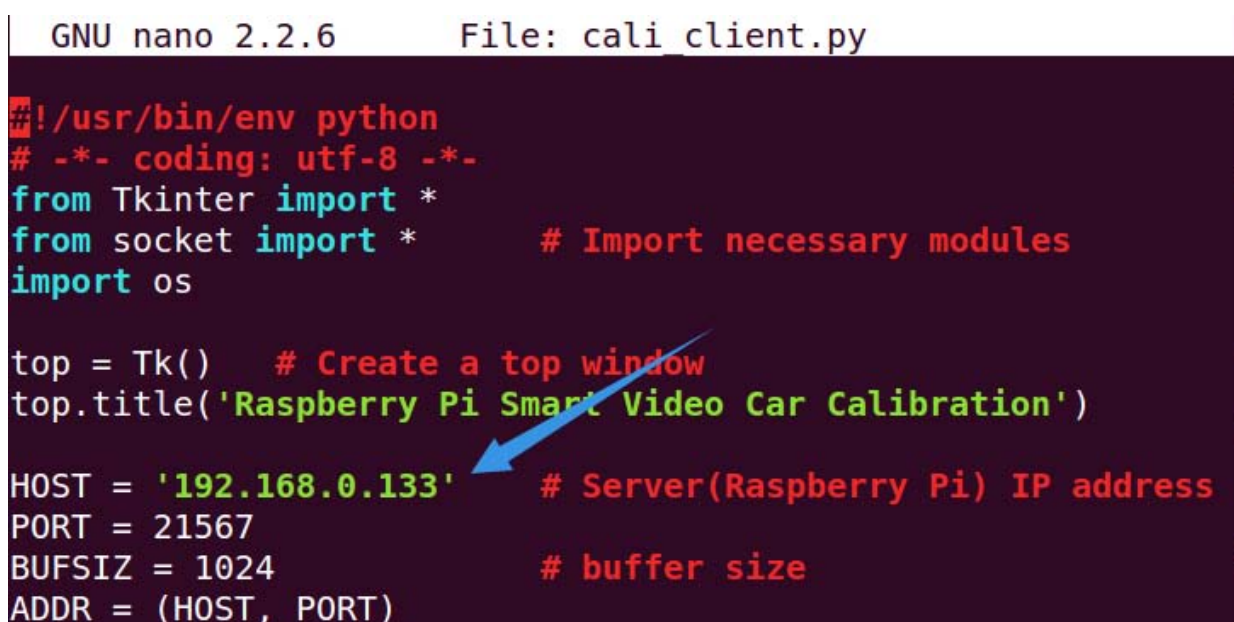
For Linux users:

Open another terminal in Linux (not via ssh on your Pi). Find the sketch downloaded and edit `client/cali_client.py`:

```
cd client/
sudo nano cali_client.py
```

```
pi@cavon:~$ cd Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/client/
pi@cavon:~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/client$ sudo nano cali_client.py
```

Find the variable of `HOST`:



```
GNU nano 2.2.6      File: cali_client.py
#!/usr/bin/env python
# -*- coding: utf-8 -*-
from Tkinter import *
from socket import *      # Import necessary modules
import os

top = Tk()      # Create a top window
top.title('Raspberry Pi Smart Video Car Calibration')

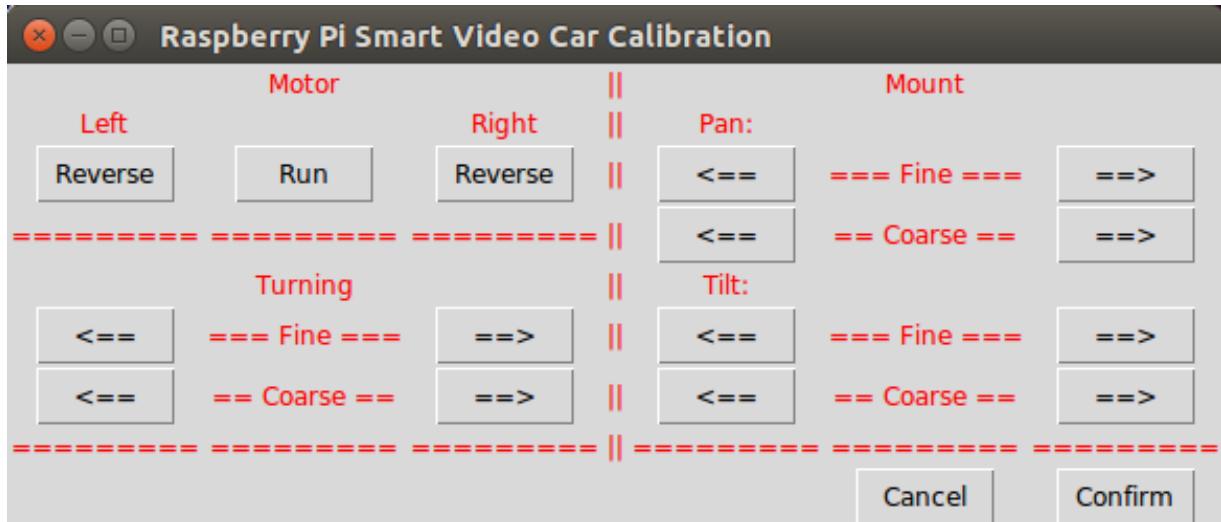
HOST = '192.168.0.133'  # Server (Raspberry Pi) IP address
PORT = 21567
BUFSIZ = 1024      # buffer size
ADDR = (HOST, PORT)
```


Enter your own address of the Raspberry Pi there. Press **Ctrl+O** to save and **Ctrl+X** exit.

Run `cali_client.py`:

```
sudo python cali_client.py
```

No matter what system your computer is running on, Linux or Windows, when you run `cali_client.py`, a window **Raspberry Pi Smart Video Car Calibration** will pop up:



In the terminal remotely connected with the Raspberry Pi, the IP address of the PC will be printed.

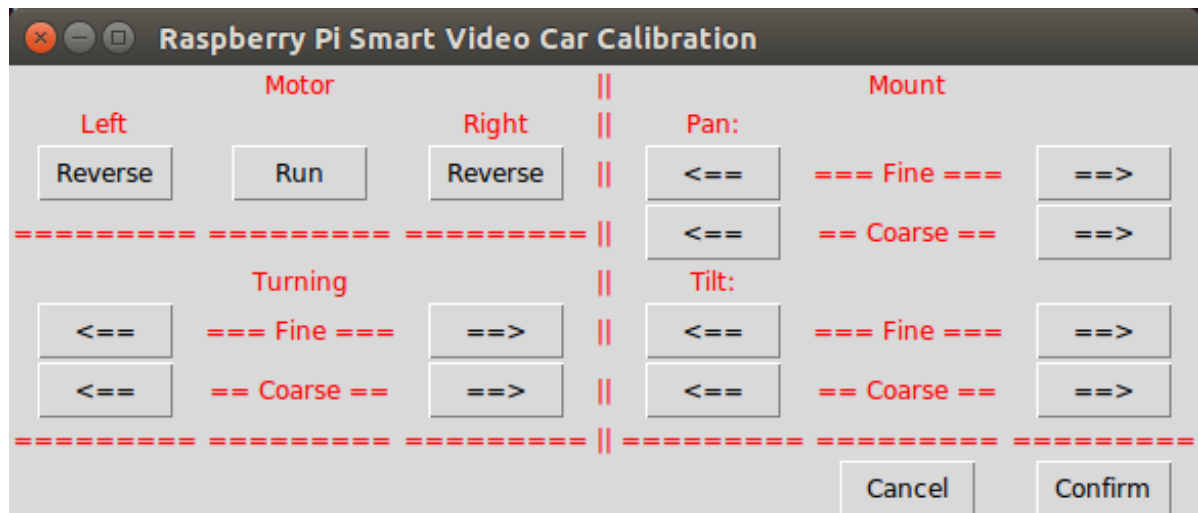
```
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_Raspberr
yPi/server $ sudo python cali_server.py
offset_x = 0
offset_y = 0
offset = 0
turning0 = True
turning1 = True
Waiting for connection...
...connected from : ('192.168.0.124', 54220)
```

Then you can start calibrating. Before that, take out your package box and place it vertically with the side face to the table. Put the car inside the box and keep it balanced. The purpose is to keep the wheels of the car off the table.

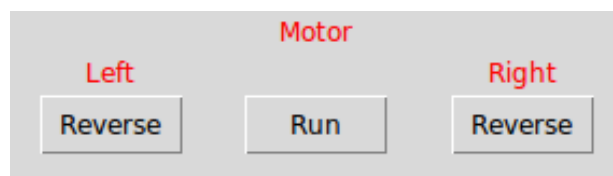
By default, the front wheels should be directly pointed towards the front; the camera on the tilt servo should be face up no matter what directions the pan servo is pointed at. If not, you may adjust the directions in **Raspberry Pi Smart Video Car Calibration**.

Start Calibration

On the calibration UI there are three sections: **Motor**, **Turning**, and **Mount**.

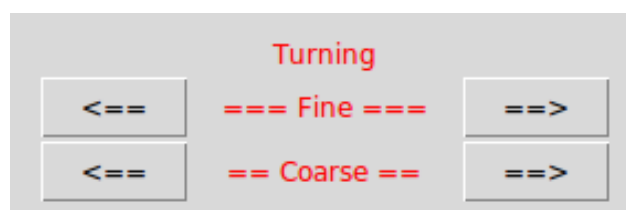


Motor Adjustment



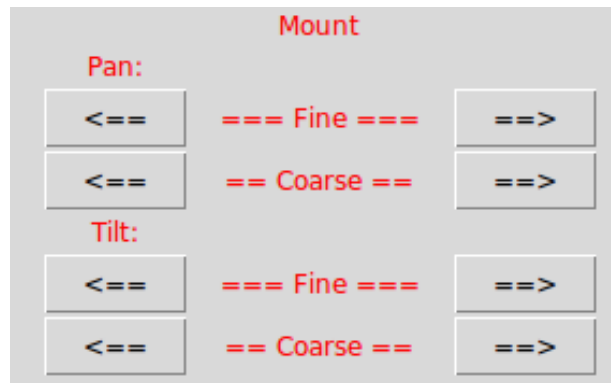
Click **Run**. The car will walk forward. Check whether both the back wheels move forward together. If either fails to do so, your wiring may be wrong. But don't worry! You don't need to rewire; just click the corresponding **Reverse** in Motor section of the Calibration window shown above. After clicking, observe whether the wheel you just adjusted is turning forward. If the reversing works normally, click **Run** again to stop the wheels from spinning.

Turning Adjustment



Currently the front wheels should be pointed at the exact front direction. But if it is not, you need to make some adjustments. In the **Turning** section in the **Calibration** window, click the left (**<==**) and right (**==>**) arrow buttons in the upper line to make fine adjustments, and those in the lower to coarsely adjust the turning direction. Keep adjusting until the wheels is oriented to the front exactly. Then you may place the car on the table and click **Run** to verify whether it runs in a straight line. If not, perform the adjustment again till it does.

Mount Adjustment

A graphical user interface titled "Mount" in red text. It is divided into two sections: "Pan:" and "Tilt:". Each section contains two rows of controls. The first row in each section is labeled "Fine" in red, and the second row is labeled "Coarse" in red. Each row has a left arrow button (labeled "<=="), a right arrow button (labeled "==>"), and a central text label. The buttons are grey with black text, and the text labels are red. The "Pan:" section has two rows, and the "Tilt:" section has two rows, for a total of four rows of controls.

Now the pan servo on the car should be pointed at the exact front direction and the camera face up. But if not, you may need to adjust them similarly. In the Mount section, there are the adjustments for the pan servo and tilt one. Also you have two kinds of adjustments, fine and coarse. Keep adjusting until they are pointed at the right direction.

After all the adjustments are done, click **Confirm**.

And the program will exit.

Then the Raspberry Pi returns to the status: **Waiting for connection...**

A screenshot of a terminal window with a dark purple background. The text "Waiting for connection..." is displayed in a light blue monospace font. A white cursor is visible at the end of the line.

Press Ctrl + C to exit.

----- **NOW YOUR CAR IS READY TO GO!** -----

Get on the Road!

Run the Server

Run the TCP server on the Raspberry Pi, and wait for the client to connect to control the car.

You need two terminals to run the server. One is to run `tcp_client.py` to receive commands to control the car, and the other to run `mjpg-streamer` to provide video streams. In Windows, you can use PuTTY to remotely log into the Raspberry Pi to start the server.

Open a terminal.

The Raspberry Pi may be still at the server directory:

```
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/server $
```

If not, go to the directory with `cd`.

```
cd ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/server
```

Then run `tcp_server.py`:

```
sudo python tcp_server.py
```

The server program on the Raspberry Pi will be running and waiting for the client to connect to the Raspberry Pi.

```
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/server $ sudo python tcp_server.py
Waiting for connection...
█
```

Open another terminal. Log in the Raspberry Pi remotely, and switch to the directory under which the program of MJPG-streamer lies:

```
cd ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/mjpg-streamer/mjpg-streamer
```

```
pi@cavon:~$ ssh pi@192.168.0.133
pi@192.168.0.133's password:
Linux raspberrypi 3.18.11+ #781 PREEMPT Tue Apr 21 18:02:18 BST 2015 armv6l

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Fri Jul 17 02:56:23 2015 from 192.168.0.118
pi@raspberrypi ~ $ cd Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/mjpg-streamer/mjpg-streamer
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/mjpg-streamer/mjpg-streamer $ █
```

Run the program:

```
sudo sh start.sh
```

Then the video data acquisition will start, like this:

```
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Pan Reset
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Tilt Reset
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Pan/tilt Reset
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Focus (absolute)
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for LED1 Mode
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for LED1 Frequency
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Disable video processing
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
mapping control for Raw bits per pixel
UVCIOC_CTRL_MAP - Error: Inappropriate ioctl for device
o: www-folder-path...: ./www/
o: HTTP TCP port.....: 8080
o: username:password.: disabled
o: commands.....: enabled
```

Type in the following address (replace *192.168.0.xxx* with the IP address of your Raspberry Pi) at the address bar of your browser (Firefox is recommended):

<http://192.168.0.xxx:8080/stream.html>

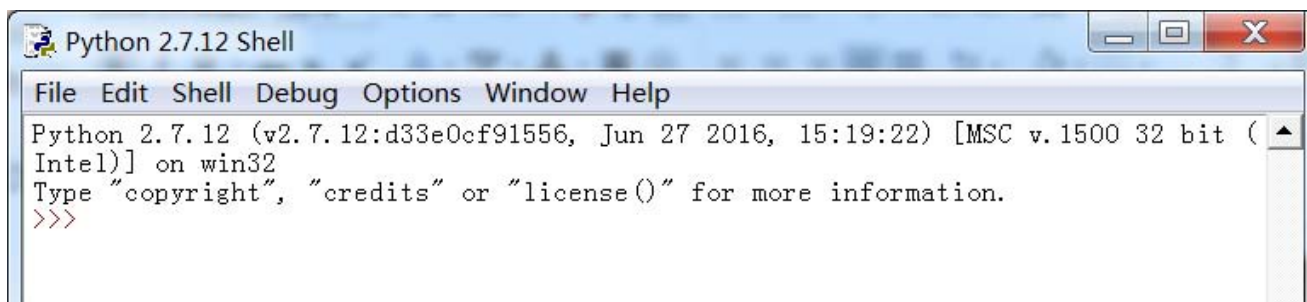
Press **Enter** and you will see the view captured by the camera displayed on the screen in a real-time manner.

Run the Client

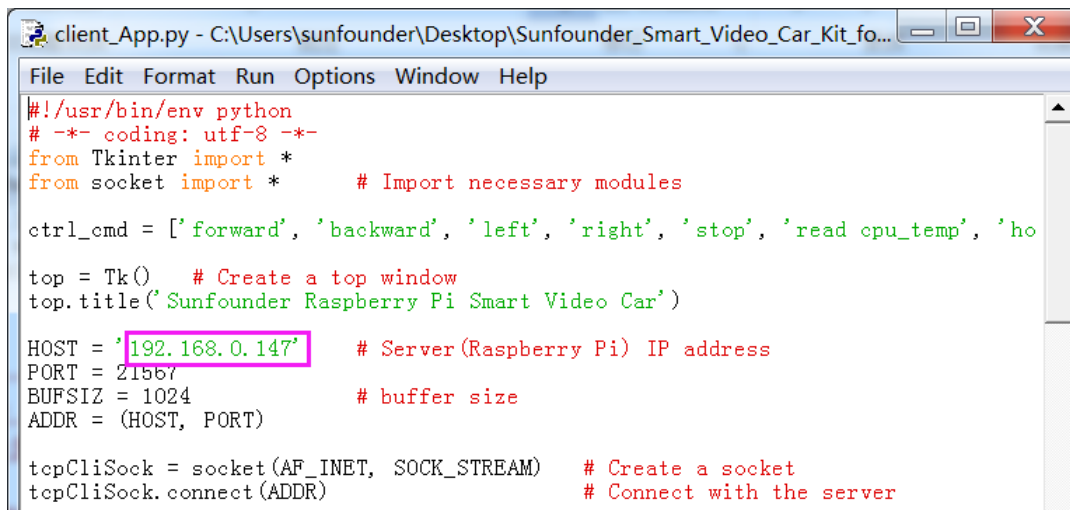
Run the client on PC to connect the server on the Raspberry Pi.

For Windows users:

Open the IDLE



Click **File** -> **Open** -> **Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi-master** -> **client** -> **client App.py** to open this file, modify the value of the HOST for the IP address of the Raspberry Pi.



```
client_App.py - C:\Users\sunfounder\Desktop\Sunfounder_Smart_Video_Car_Kit_fo...
File Edit Format Run Options Window Help
#!/usr/bin/env python
# -*- coding: utf-8 -*-
from Tkinter import *
from socket import *      # Import necessary modules

ctrl_cmd = ['forward', 'backward', 'left', 'right', 'stop', 'read cpu_temp', 'ho

top = Tk()      # Create a top window
top.title('Sunfounder Raspberry Pi Smart Video Car')

HOST = '192.168.0.147'    # Server(Raspberry Pi) IP address
PORT = 21567
BUFSIZ = 1024           # buffer size
ADDR = (HOST, PORT)

topCliSock = socket(AF_INET, SOCK_STREAM)    # Create a socket
topCliSock.connect(ADDR)                    # Connect with the server
```

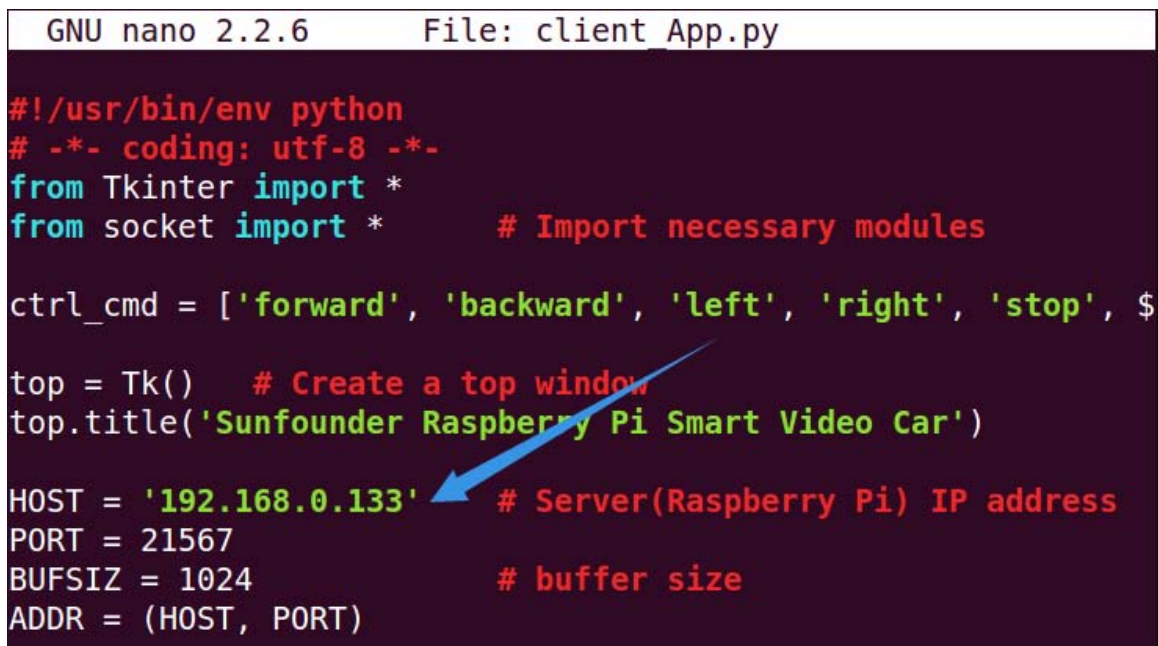
After modification, save the file and select **Run** -> **Run Module**.

For Linux users:

Open another terminal again. But do not log into the Raspberry Pi remotely in this terminal. Go to the client with `cd` and edit the file `client.py`:

```
pi@cavon:~$ cd Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi
/client/
pi@cavon:~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi/cli
ent$ sudo nano client_App.py
```

Similar to changes in calibration, change the IP address in client program in your PC:



```
GNU nano 2.2.6      File: client App.py

#!/usr/bin/env python
# -*- coding: utf-8 -*-
from Tkinter import *
from socket import *      # Import necessary modules

ctrl_cmd = ['forward', 'backward', 'left', 'right', 'stop', $

top = Tk()      # Create a top window
top.title('Sunfounder Raspberry Pi Smart Video Car')

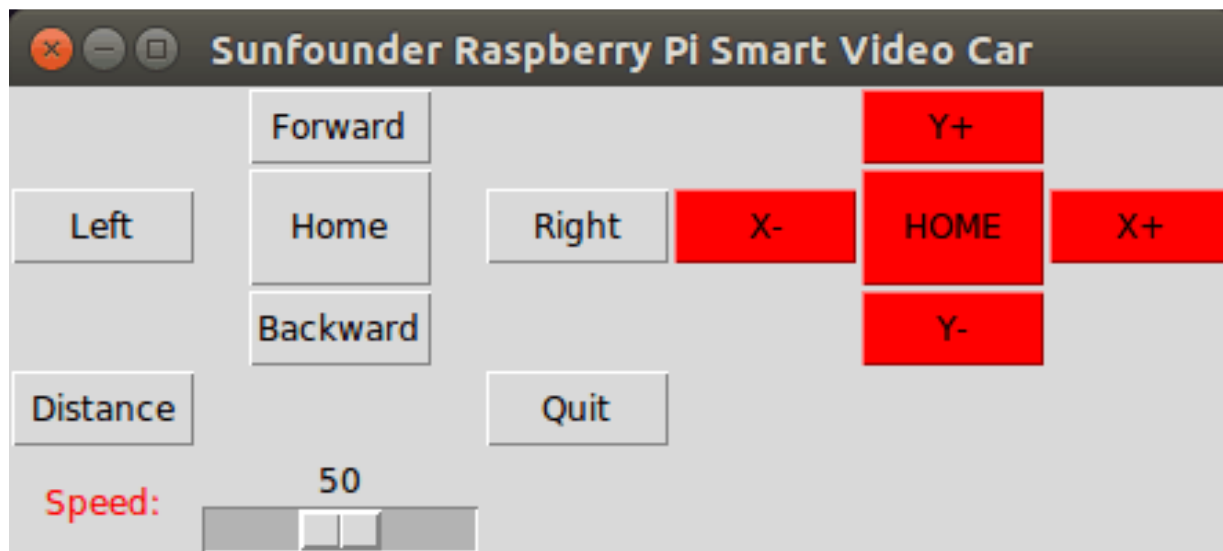
HOST = '192.168.0.133'    # Server(Raspberry Pi) IP address
PORT = 21567
BUFSIZ = 1024           # buffer size
ADDR = (HOST, PORT)
```

After the alteration, press **Ctrl** + **O** and save and **Ctrl** + **X** to exit.

Then run the client program:

```
sudo python client_App.py
```

No matter what system your computer is running on, Linux or Windows, when you run the `client_App.py`, the following window will appear on your screen:



You can click buttons such as **Forward** and **Backward** to control the car moving remotely. Or click **X+**, **X-**, **Y+**, and **Y-** to control the coverage of the camera.

Note:

*The server program must be run **before** you run the client program. Some settings must be completed for the server before the service is done. A communication endpoint needs to be created for the server to "listen" to requests from the client. Take the server as a receptionist or an operator of the bus phone in a company. Once the phone and device installation is completed and the receptionist or operator is in place, the service begins.*

v. Program Analysis and Explanation

Abstract

From the perspective of software, the smart car is of C/S structure. The TCP server program is run on Raspberry Pi to listen to the command from the client and control the car accordingly. The client program is run on the PC and connected with the server through the TCP, which provides the user with a graphical user interface (GUI) to conveniently control the Raspberry Pi remotely. Both the client and server programs are written in Python.

Make sure that the circuit is connected properly. Power the smart car, log in the Raspberry Pi remotely, go to the directory *Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi* and check the files under it.

```
cd ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi
ls
```

```
pi@raspberrypi ~/Sunfounder_Smart_Video_Car_Kit_for_RaspberryPi $ ls
client  datasheet  html_server  i2cHelper.py  mjpg-streamer  README.md  server
```

You can see seven files under the directory: **client**, **datasheet**, **html_server**, **i2cHelper.py**, **mjpg-streamer**, a file **README.md**, and **server**.

Wherein,

client, the client run on your PC,

datasheet contains some PDF files about the chip (you need to view them on PC),

html_server, the web server run on the Raspberry Pi for Android app client,

i2cHelper, a simple script to help you set up i2c on the Raspberry Pi,

mjpg-streamer, the camera driver to acquire and upload images,

README.md, an introduction file with update information,

server, the server run on the Raspberry Pi for the client on your PC.

Introduction of Socket

The C/S-structure program of the SunFounder Raspberry Pi-based Smart Car is written based on the socket module of the Python language. Socket wraps and applies the TCP/IP and is used to describe IP address and port. Also it is a network data structure for computer. The socket module should be created before the communication of network applications. If the socket can be said to be the plug of a telephone, which is the lowest layer of communication, then the combination of IP address and ports can be said to be that of area code and phone numbers. Only having the hardware for a phone call making is not enough. You still need to know whom and where to call. An Internet address is composed of the essential IP address and port for network communication.

1. Server

Here we provide a pseudocode which creates a universal TCP server for explanation. Note that this is just one of the methods for server design. After you have a good knowledge about it, you can alter the pseudocode as you want:

```

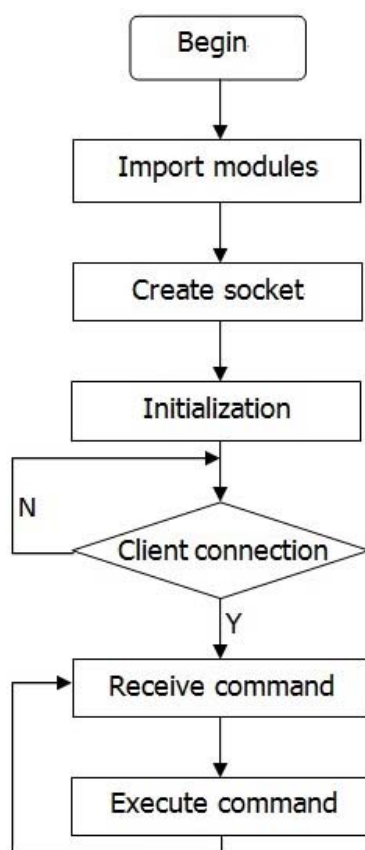
s = socket( )           # Create a socket for the server.
s.bind( )               # Bind the address to the socket.
s.listen( )             # Listen to the connection.
inf_loop:               # Indefinite loop of the server.
    c = s.accept( )      # Accept the connection from the client.
comm_loop:              # Communication loop.
    c.recv( )/c.send( )  # Dialog (receiving or sending data)
c.close( )               # Close the socket of the client.
s.close( )               # Close the socket of the server (optional).

```

All kinds of socket can be created via the function `socket.socket( )` and then bound with IP address and port by the function `bind( )`. Since TCP is a connection-oriented communication system, some settings need to be completed before the TCP server starts operation. The TCP server must "listen" to connections from the client.

After the settings are done, the server will enter an indefinite loop. A simple, like single-thread, server will call the function `accept( )` to wait for the coming connection. By default, the function `accept( )` is a blocking one, which means it is suspended before the connection comes. Once a connection is received, the function `accept( )` returns a separate client socket for the subsequent communication. After the temporary socket is created, communication begins. Both the server and client use the new socket for data sending and receiving. The communication does not end until either end closes the connection or sends a null character string.

Process Diagram of Server Program



2. Client

It is easier to create a TCP client than to do a server. Take the following pseudocode:

```
c = socket( )           # Create a client socket.
c.connect( )            # Try to connect a server.
comm_loop:              # Communication loop.
    c.send( )/c.recv( )  # Dialog (sending out and receiving data)
c.close( )              # Close the client socket.
```

As mentioned above, all sockets are created via the function `socket.socket()`. Then, the function `connect()` can be called to connect the server. After the connection is built, the dialog between the client and the server is enabled. When the dialog ends, the client can close the socket and the connection.

Introduction of Tkinter

Developed based on Tkinter, our client program carries graphical interfaces. Tkinter is a GUI widget set for Python. We can develop application programs with graphical interfaces fast by Python language based on it. It is quite easy to use Tkinter. All you have to do is to import the module into Python.

To create and run a GUI program, take the following steps:

- Import the Tkinter module (by `import Tkinter` or `from Tkinter import *`).
- Create a top window object to contain the whole GUI program.
- Create the GUI module needed on the object and enable the functions.
- Connect the GUI modules with the code at the system back-end.
- Enter the main event loop.

Take a simple GUI program:

Create a file `Tk_test.py` under the path `/home`:

```
touch Tk_test.py
```

Add executable privilege to the file:

```
chmod +x Tk_test.py
```

Open the file:

```
vim Tk_test.py
```

Type in the following code:

```
#!/usr/bin/env python
from Tkinter import *

top = Tk()           # Create a top window
top.title('Sunfounder.com')

label = Label(top, text='Hello Geeks !', fg='blue') # Create a label and set its
foreground color as blue
label.pack()         # layout
```

```
top.mainloop() # main loop
```

Save the code and exit.

Run

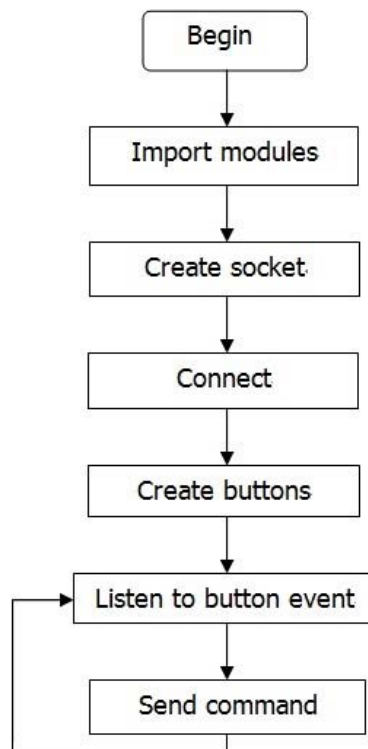
```
./Tk_test.py
```

Then the following picture will appear on your screen:



Click  to close the program.

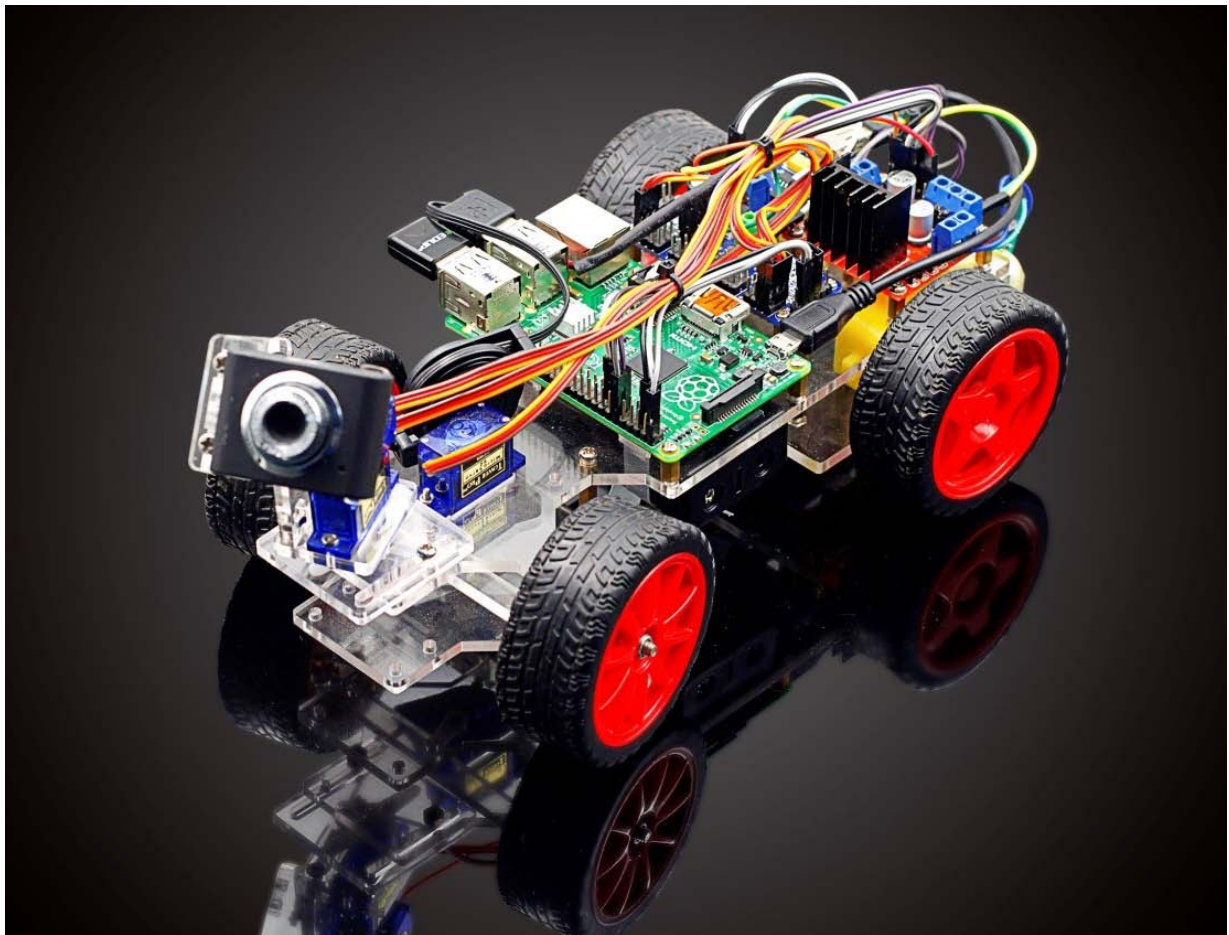
Process Diagram of Client Program



Summary

In this manual, having learned the related components for building the car kit, you've gone through the assembly of the mechanical parts and electrical modules with the knowledge of Raspberry Pi as well as a brief introduction of the key parts like servo, Wi-Fi adapter, etc. Also you've got a lot of software and coding, which lays a solid foundation for your future journey of exploring open-source field.

The **SunFounder Smart Video Car for Raspberry Pi** is not only a toy, but more a meaningful development kit for Raspberry Pi. After all the study and hands-on practice of the kit, you should have a better understanding of Raspberry Pi. Now, get started to make better work!



Copyright Notice

All contents including but not limited to texts, images, and code in this manual are owned by the SunFounder Company. You should only use it for personal study, investigation, enjoyment, or other non-commercial or nonprofit purposes, under the related regulations and copyrights laws, without infringing the legal rights of the author and relevant right holders. For any individual or organization that uses these for commercial profit without permission, the Company reserves the right to take legal action.